

LAB 1: INTRODUCTION TO DIFFERENT KINDS OF TOOLS USED IN DESIGNING SOFTWARE ENGINEERING DESIGNS

Introduction

Software design tools are the applications that assist in the process of planning and structuring a software system before coding. These tools help transform user requirements into a blueprint by creating diagrams, models, and documentation for components, interfaces, and data flow, which improves efficiency and collaboration among developers. Example: Entity Relationship Diagram, Flowchart, Data Flow Diagram, User Interface designs tools, modelling tools etc.

The three different websites we visited. These are:

1. Draw.io
2. Lucidchart
3. StarUML

1. Draw.io

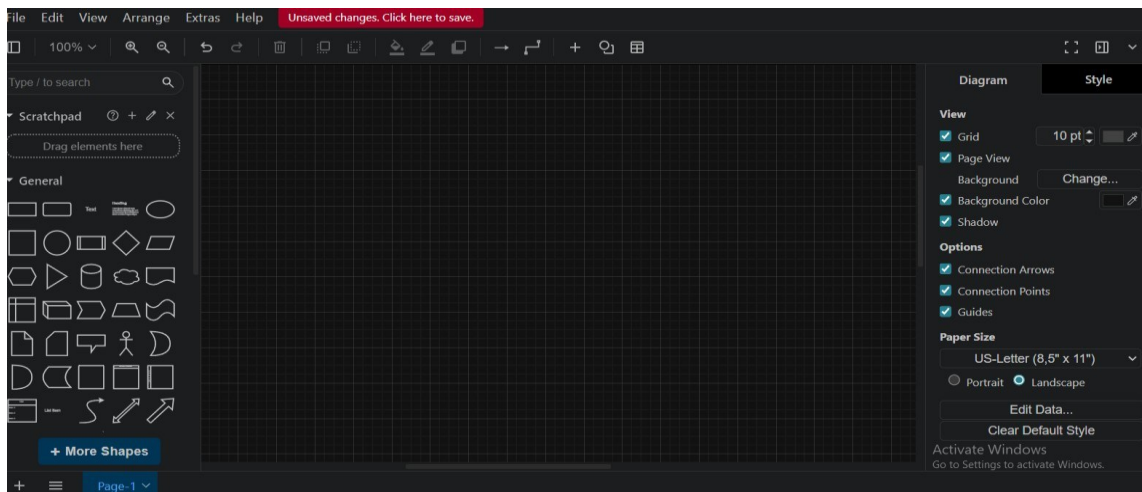


Fig: Draw.io Screen

Draw.io (diagrams.net) is a free and open-source tool for creating a wide range of diagrams, including flowcharts, network diagrams, and organizational charts. The platform features a drag-and-drop interface, a large library of shapes, and integrations with various cloud storage providers.

We draw various tools in draw.io like this:

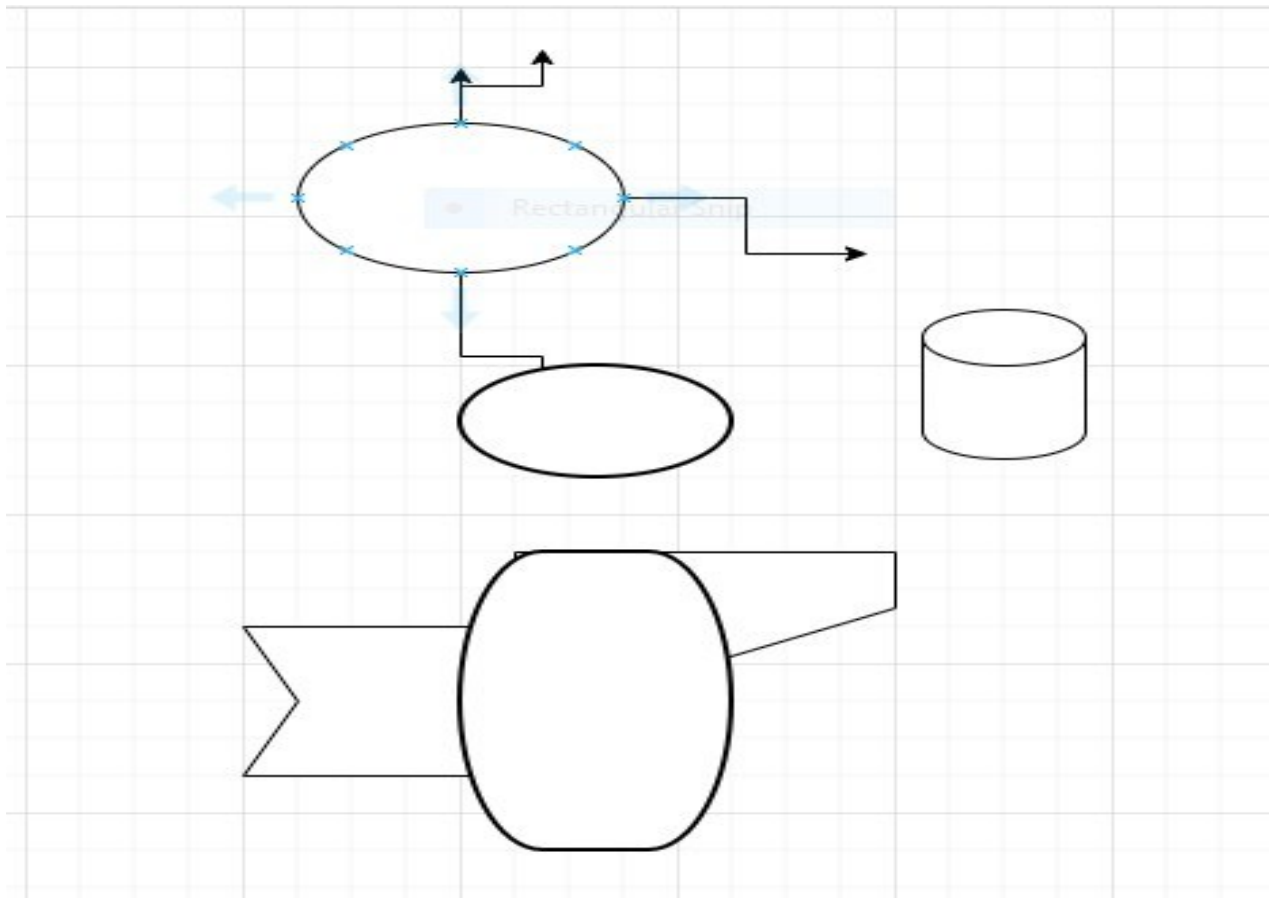


Fig: drawing using various shapes

2.Lucid Chart

Lucid chart is a web-based diagramming application that allows teams to collaborate in real time to create and share visual documents like flowcharts, org charts, and technical diagrams. It is used to improve processes, visualize systems, and enhance team collaboration through features like AI-powered tools, data linking, and integration with other apps like Google Workspace, Microsoft Office, and Asana.

We make various charts and used the different tools are as given below:

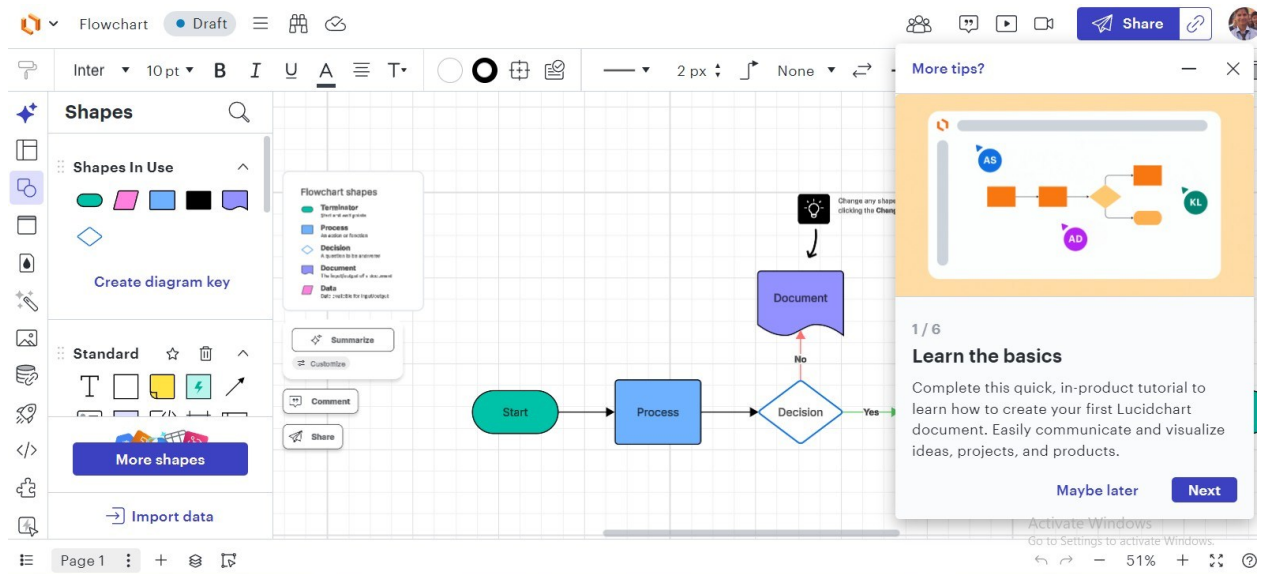


fig: Screen of Lucid Chart for Flowchart

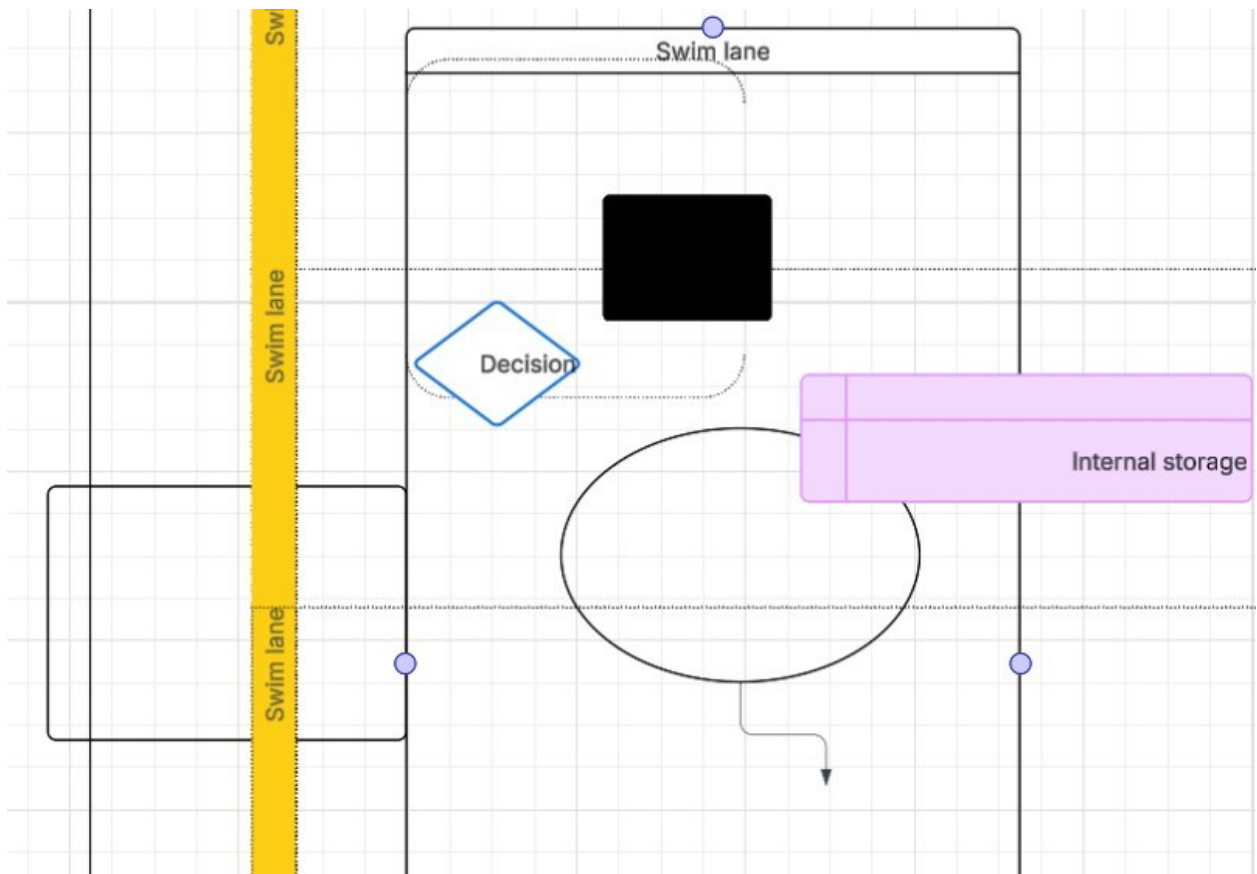


fig: drawing using different tools.

3.Startuml

Star UML is a sophisticated modeling tool that supports creating various software engineering diagrams, primarily using the Unified Modeling Language (UML). It is used by software architects, developers, and students to design, document, and visualize software systems through its intuitive interface and functionality. The tool also supports other diagrams like ER diagrams, flowcharts, and mind maps.

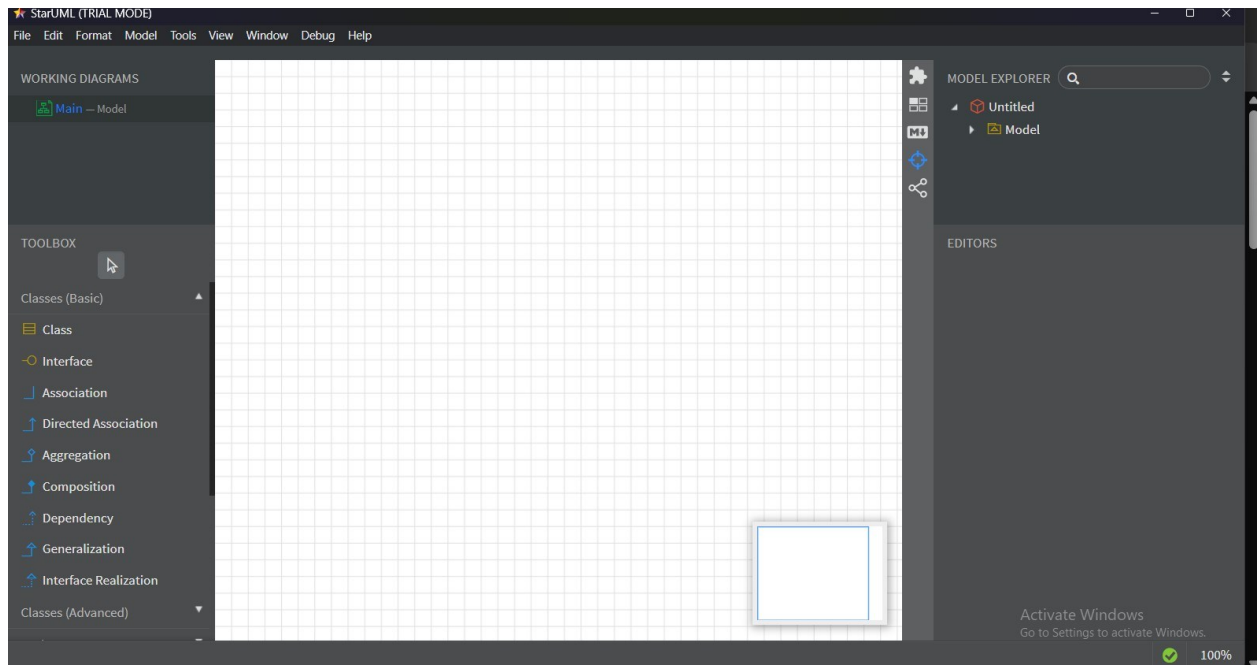


fig: Screen of StarUML

Conclusion

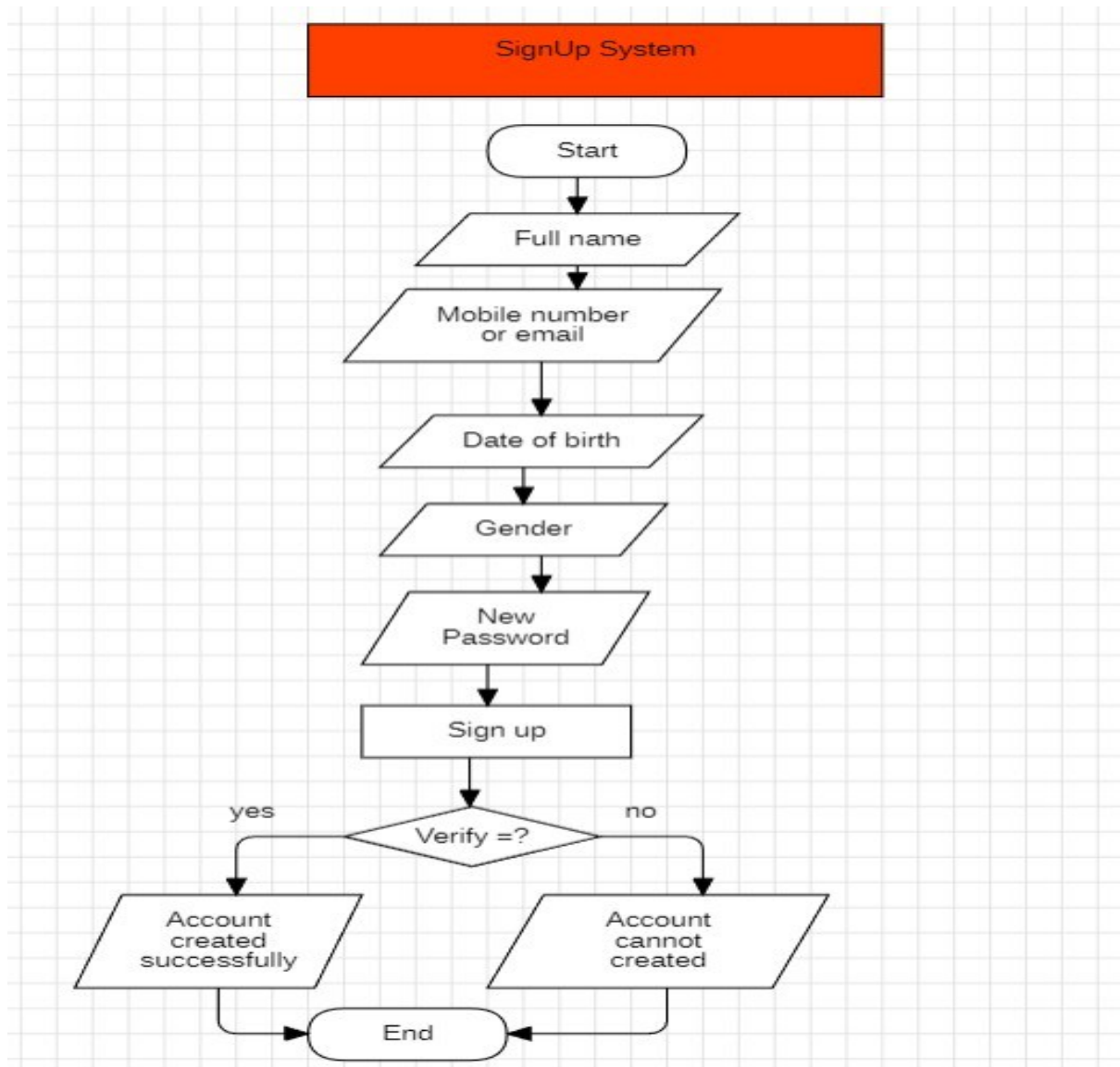
We learn about various designing tools used in design of software. Draw.io, LucidChart and StarUML each have its own features and functions. We mostly use StarUML in this lab.

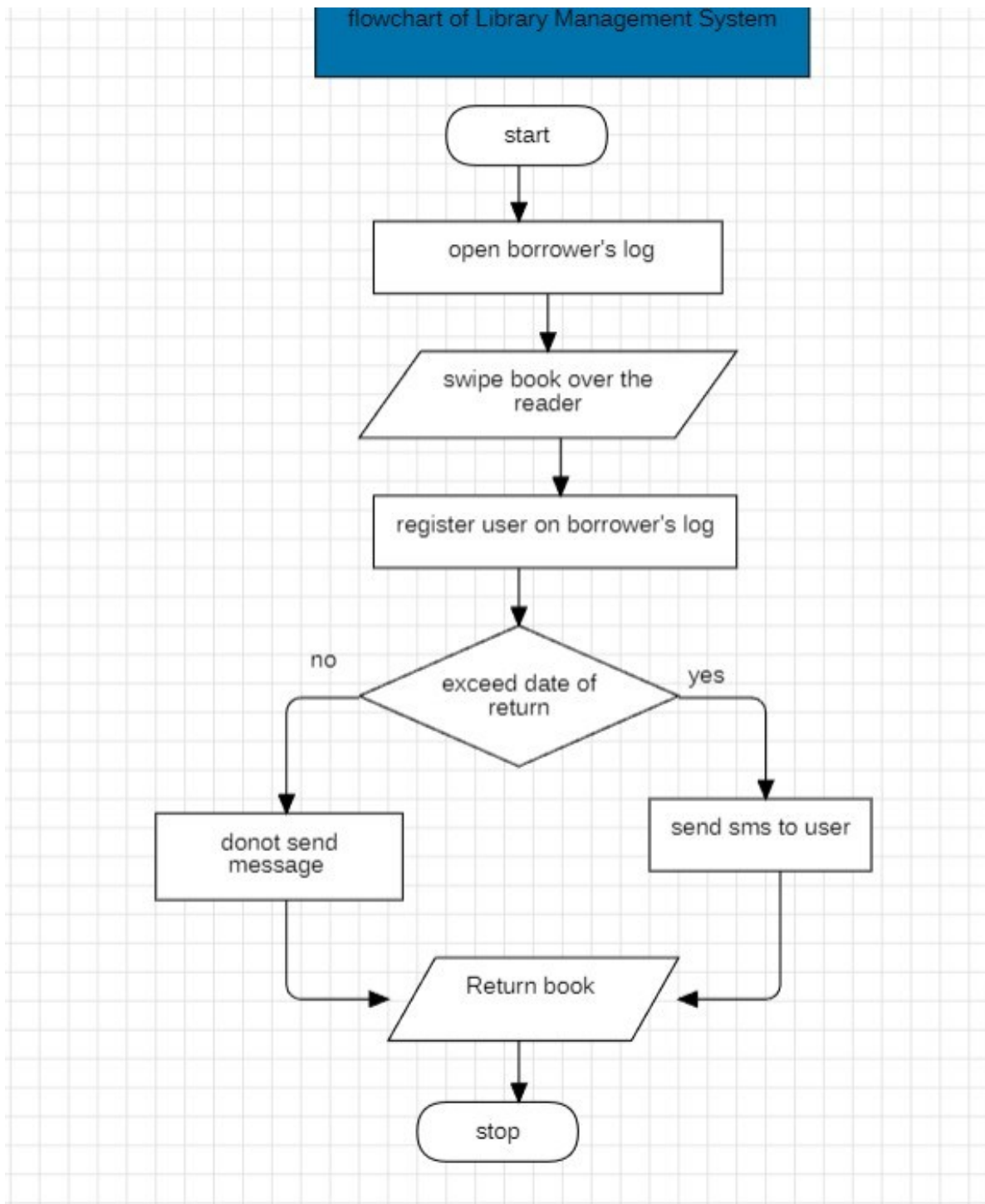
LAB 2: DESIGNING FLOWCHART USING CASE TOOLS.

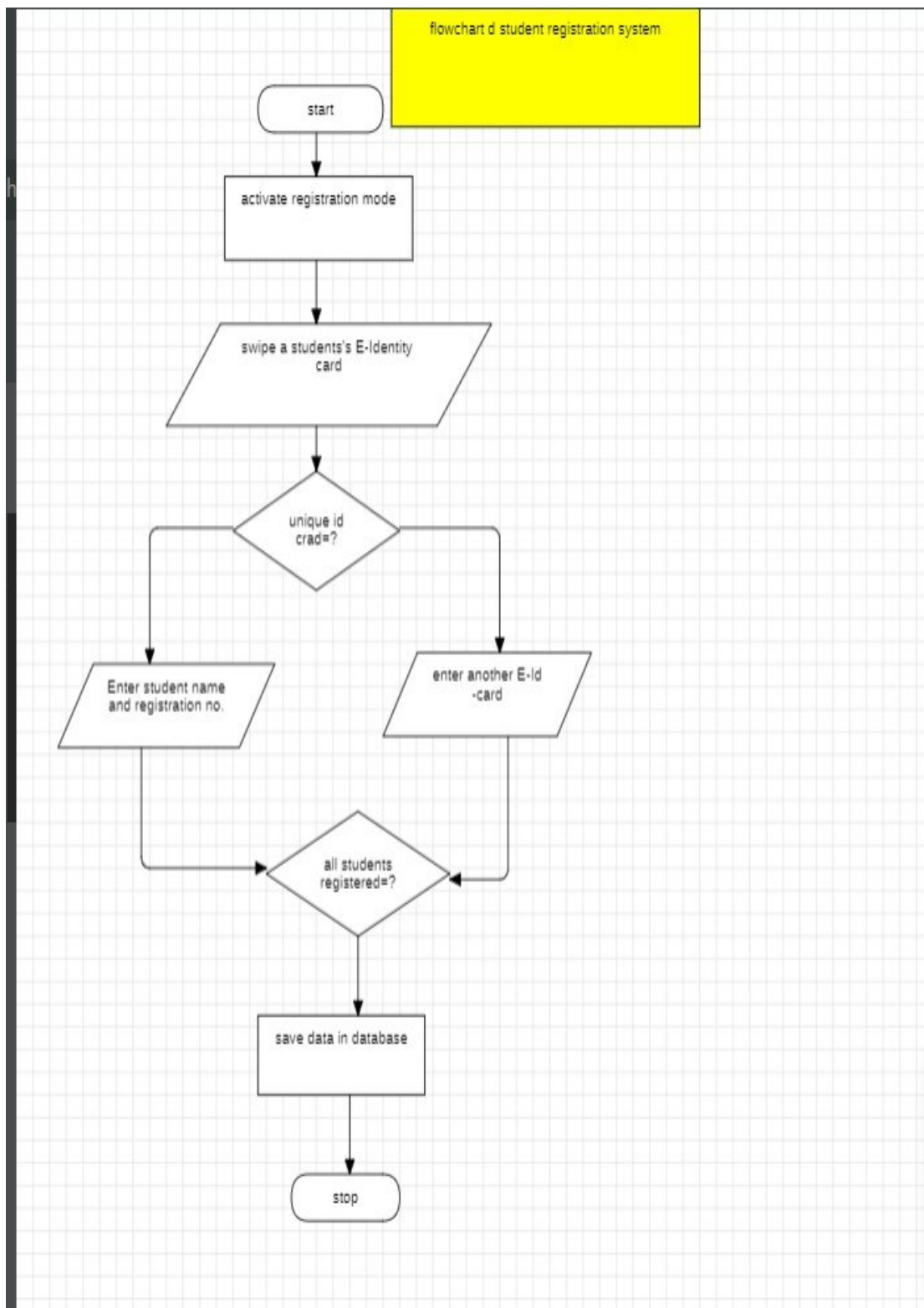
Introduction

CASE Tools: CASE (Computer Aided Software Engineering) tools are software programs that automate and support various stages of the software development lifecycle (SDLC), such as requirements analysis, design, coding, testing, and maintenance.

Flowchart: It is pictorial representation of an algorithm. It shows the steps in graphical way We have made different flowcharts in lab a







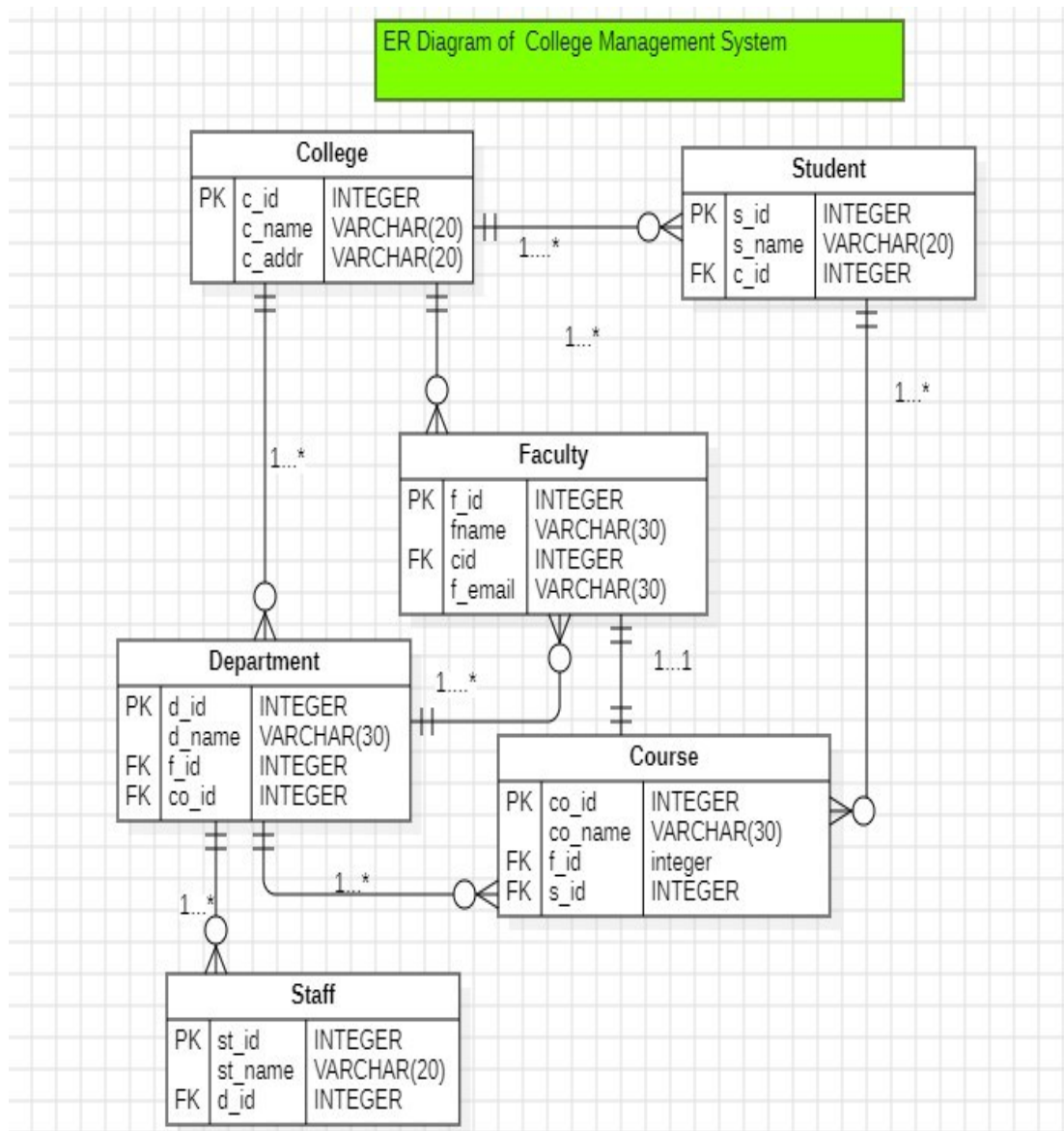
Conclusion

In this way We learned to design different flowcharts using different CASE tools. This makes flowcharts faster and easier to design.

LAB 3: DESIGN ER DIAGRAM USING CASE TOOLS

Introduction

ER Diagram: Entity Relationships Diagram is a visual blueprint for databases, using symbols like rectangles (entities), ovals (attributes), and diamonds (relationships) to map out real-world objects (people, places, things) and how they connect, forming the logical structure of a system and guiding database design, development, and debugging.



Conclusion

In this way we have prepared ER diagram.

LAB 4: Design Sequence Diagram Using CASE Tools

Introduction

Sequence Diagram: A sequence diagram is a type of UML (Unified Modeling Language) diagram that shows how objects in a system interact with each other over time, focusing on the chronological order of messages and actions to achieve a specific goal or use case, visualizing the flow of control and data between components like a user, database, and server. It uses lifelines (vertical lines) for objects and horizontal arrows for messages, making complex system behaviors, like a login process or ATM withdrawal, easy to understand.

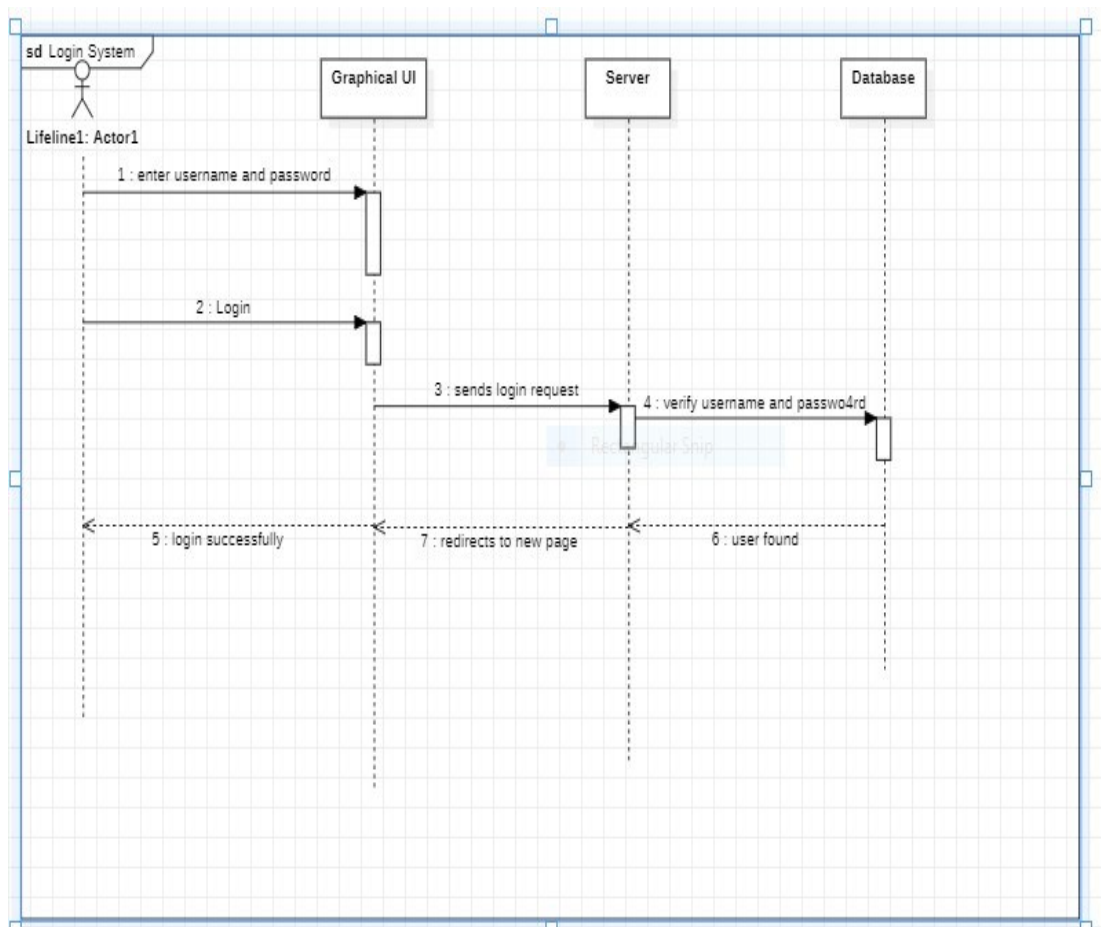


Fig:Sequence Diagram of Login System

Conclusion

In this way we have learned Sequence Diagram designing.

LAB 5: DESIGNING USE CASE DIAGRAM USING CASE TOOLS

Introduction

Use case Diagram: A Use Case Diagram visually maps user interactions (Actors) with a system's functions (Use Cases), showing what the system does from a user's perspective, not how, highlighting goals, actors, and their relationships (like include, extend, generalization) to define system requirements during analysis and design.

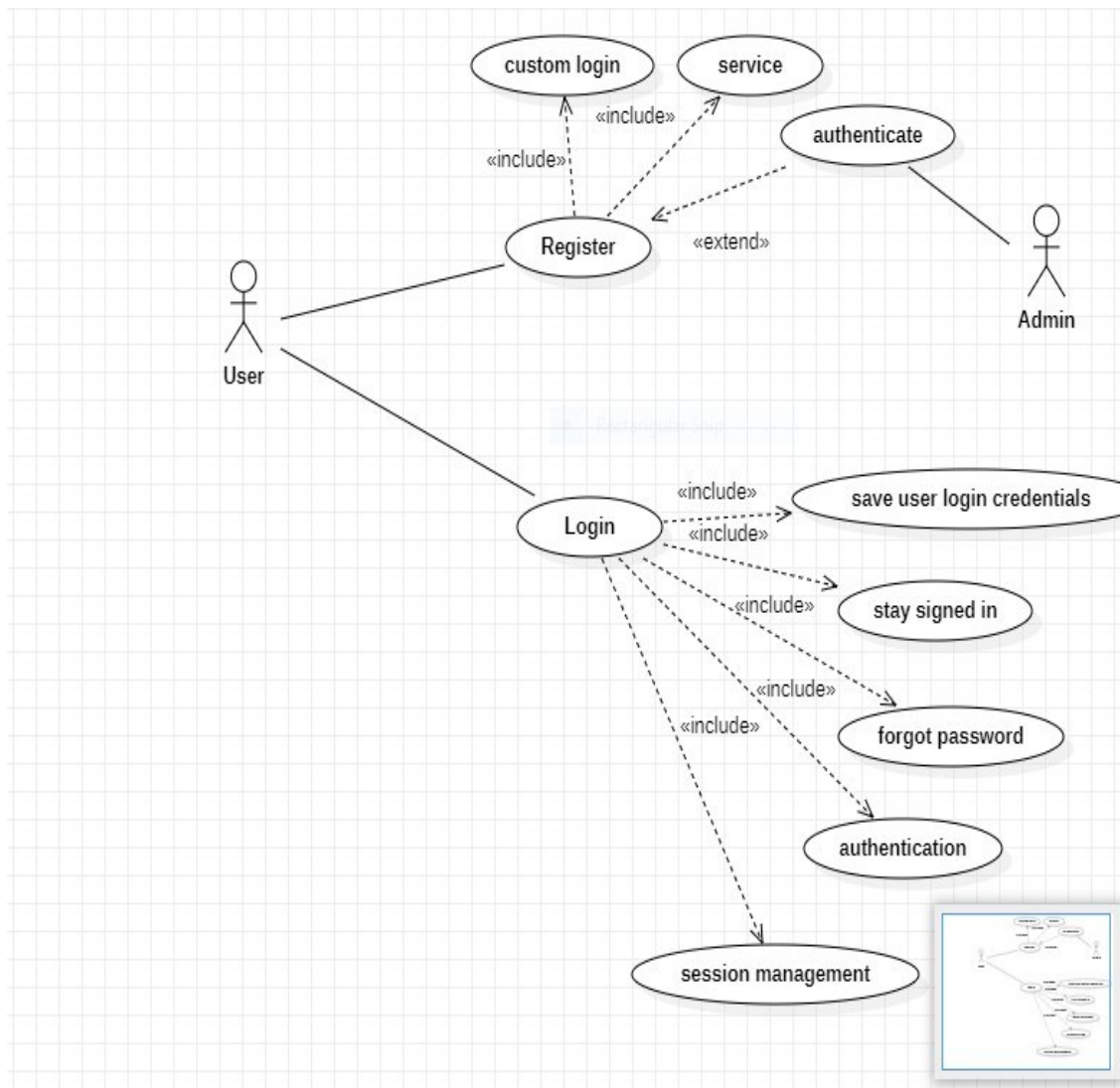


Fig: Use Case Diagram of Login System

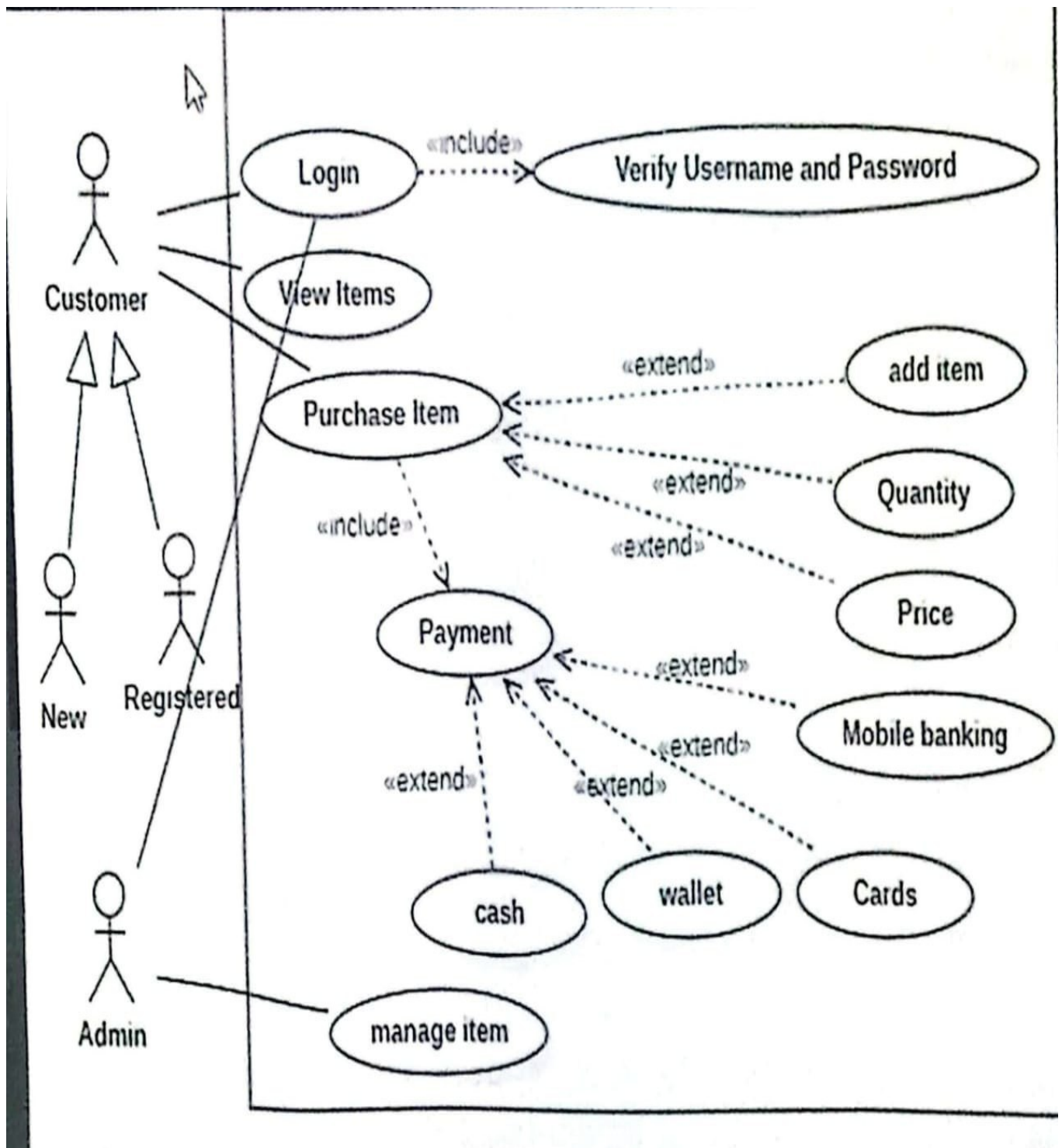


Fig: Use Case Diagram of Online Shopping System

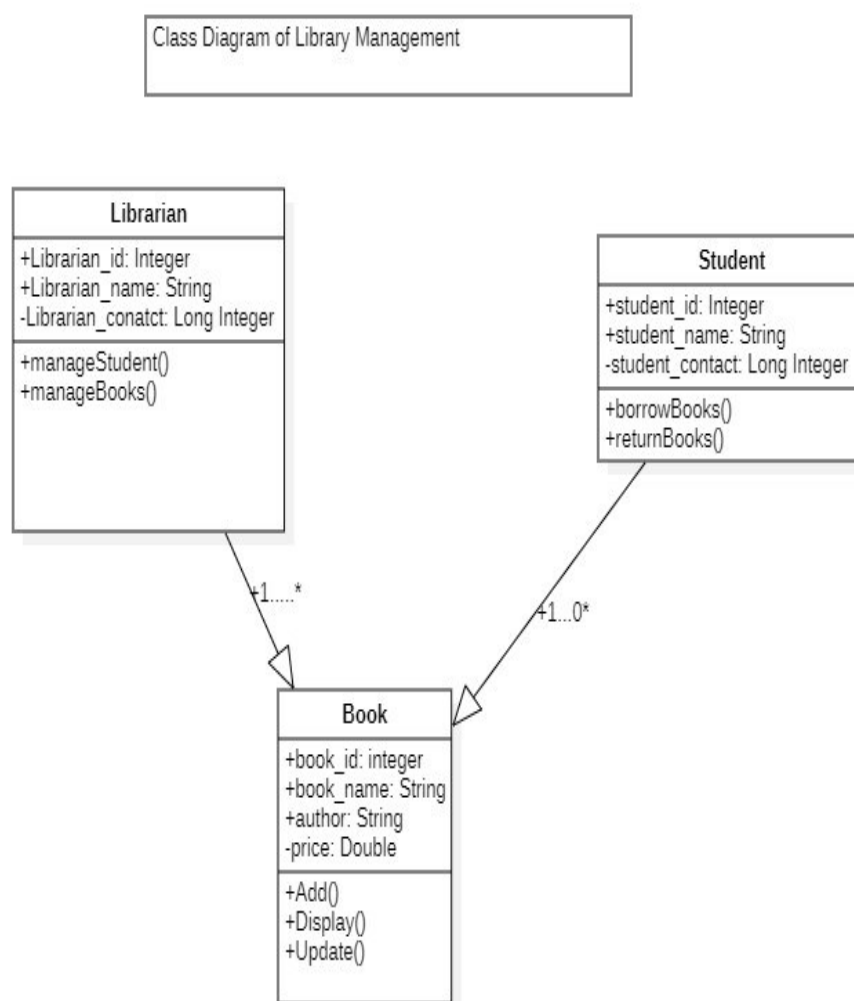
Conclusion

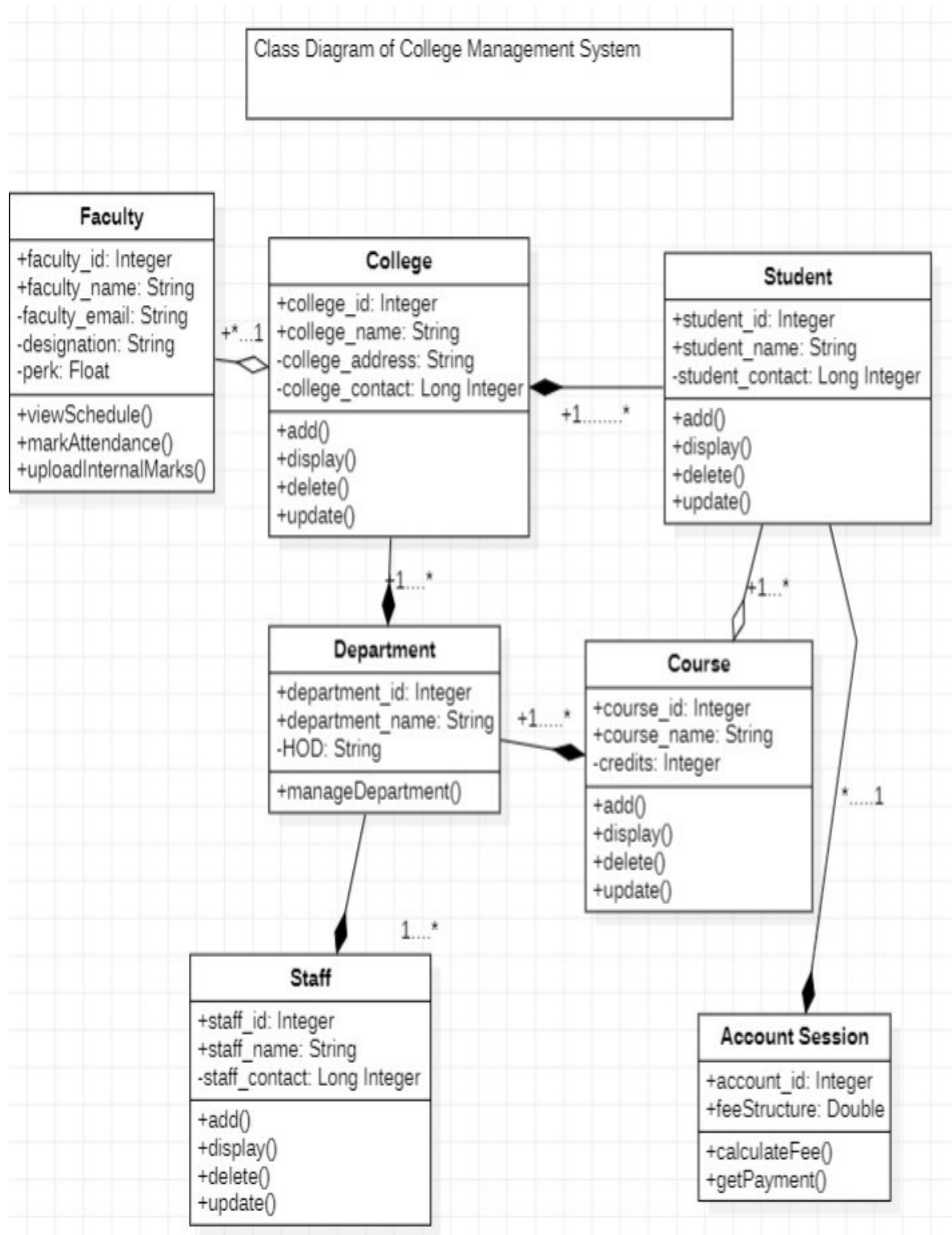
In this way we have learned Use Case Diagram design.

LAB 6: DESIGNING CLASS DIAGRAM USING CASE TOOLS

Introduction

Class Diagram: A class diagram, part of UML (Unified Modeling Language), visually maps a system's static structure by showing classes, their attributes (data), operations (methods/functions), and the relationships (like inheritance, association) between them, serving as a blueprint for software design, documentation, and even direct code generation in object-oriented programming.





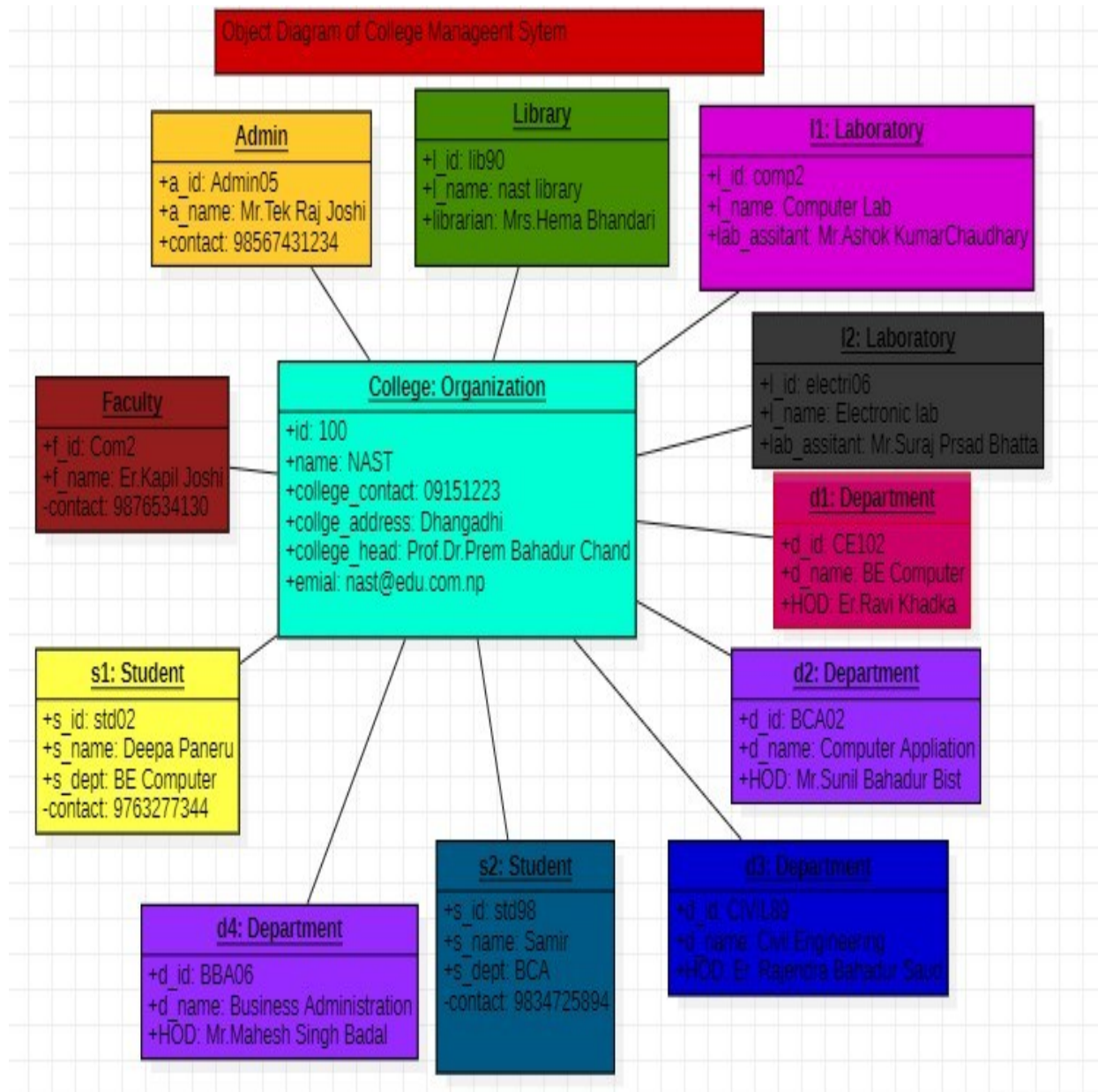
Conclusion

In this way we have learned design Class Diagram.

LAB 7: DESIGNING OBJECT DIAGRAM USING CASE TOOLS

Introduction

Object Diagram: An object diagram is a UML diagram that shows a snapshot of a system's structure at a specific moment, illustrating concrete object instances and the links between them, rather than abstract classes.



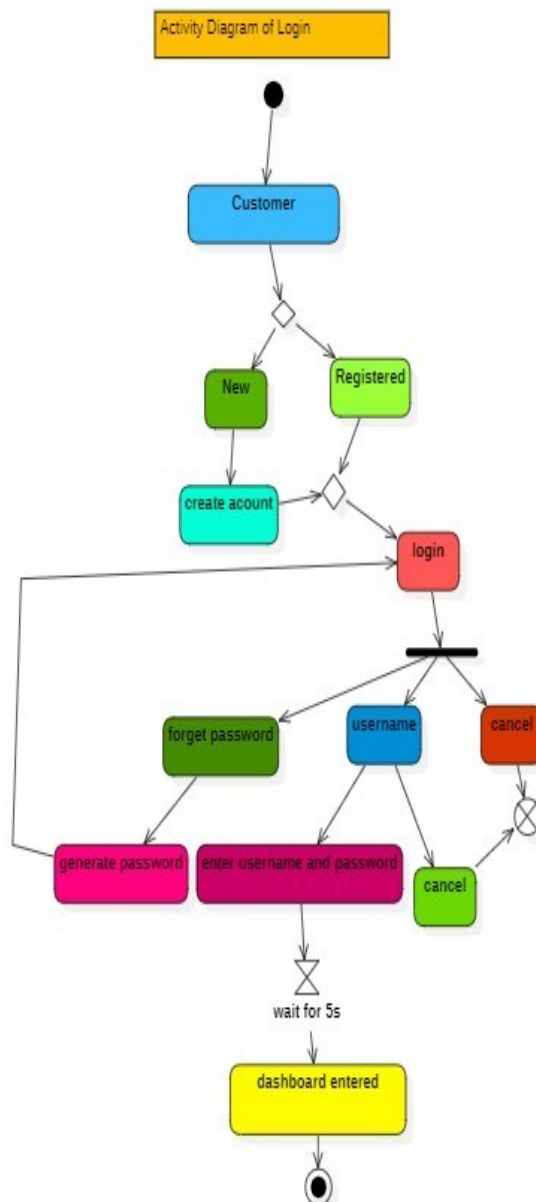
Conclusion

In this way We have learned to design Object Diagrams.

LAB 8: DESIGNING ACTIVITY DIAGRAM USING CASE TOOLS

Introduction

Activity Diagram: It is a UML that visually maps out the step-by-step flow of actions, behaviors, and decisions within a system, process, or workflow, acting as an advanced flowchart that can show concurrent activities, data flow, and different paths.



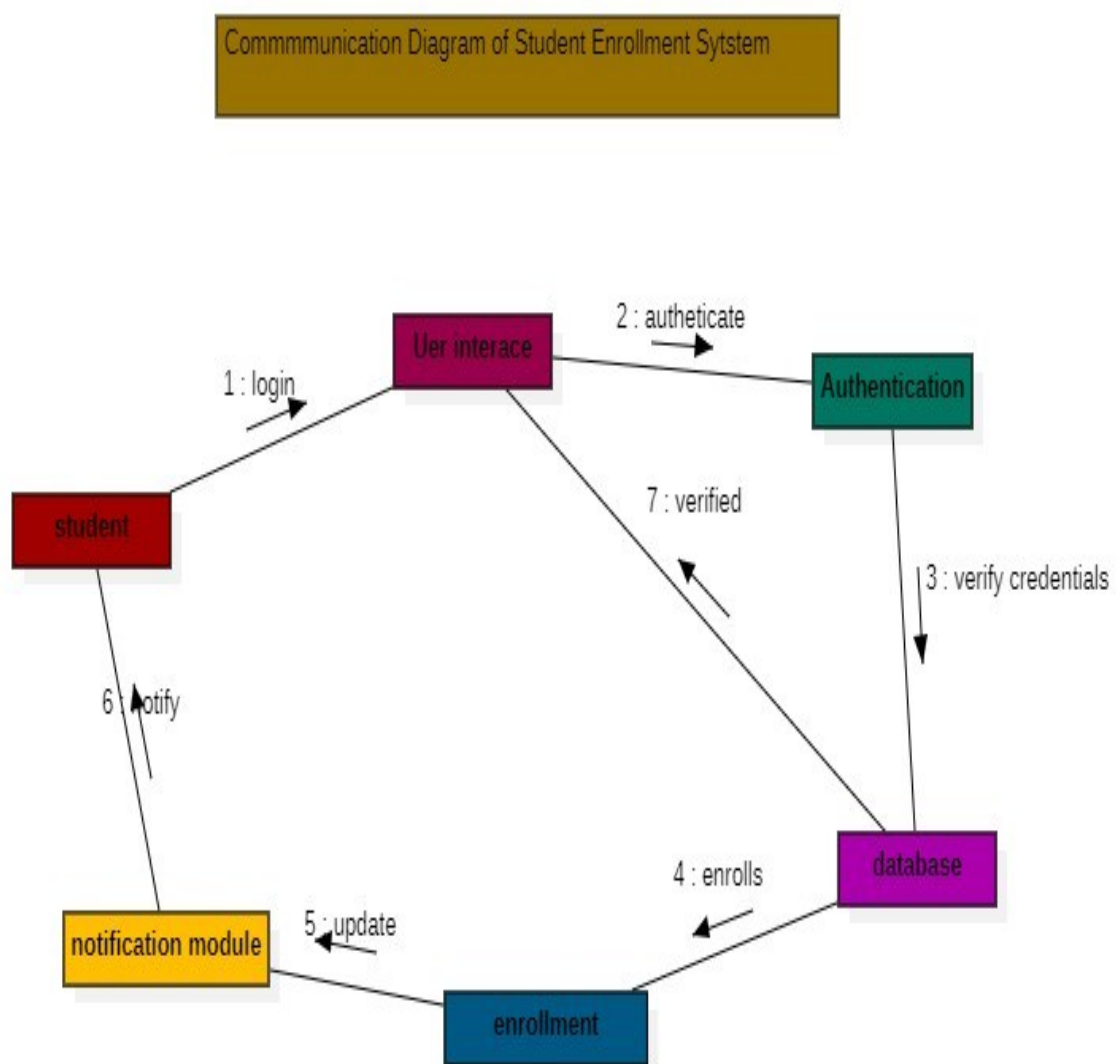
Conclusion

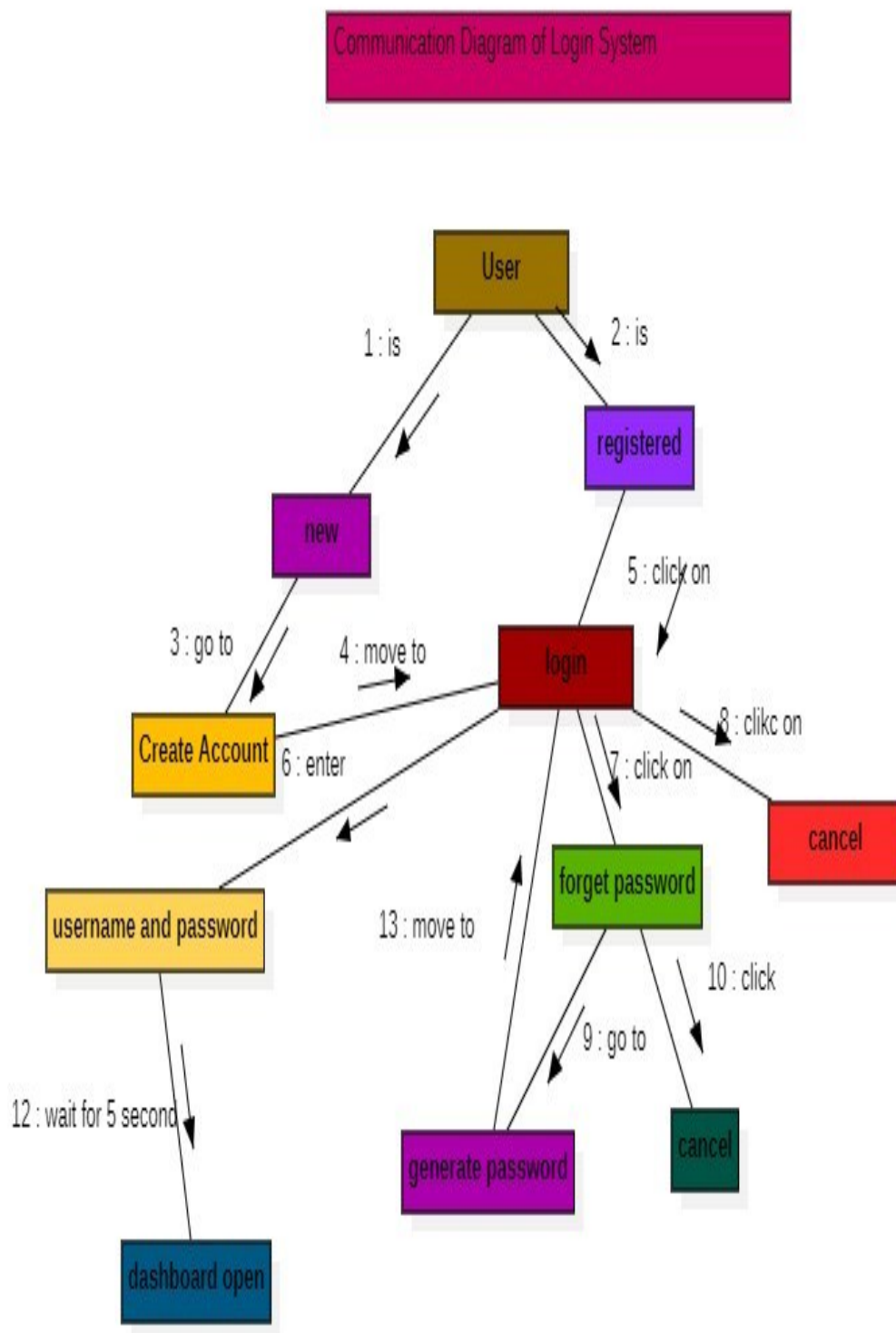
In this way We have learned to design Activity Tools.

LAB 9: DESIGNING COMMUNICATION DIAGRAM USING CASE TOOLS

Introduction

Communication Diagram: It is a UML that shows how objects interact by exchanging messages or interacting with each other's focusing on object relationships and message paths .





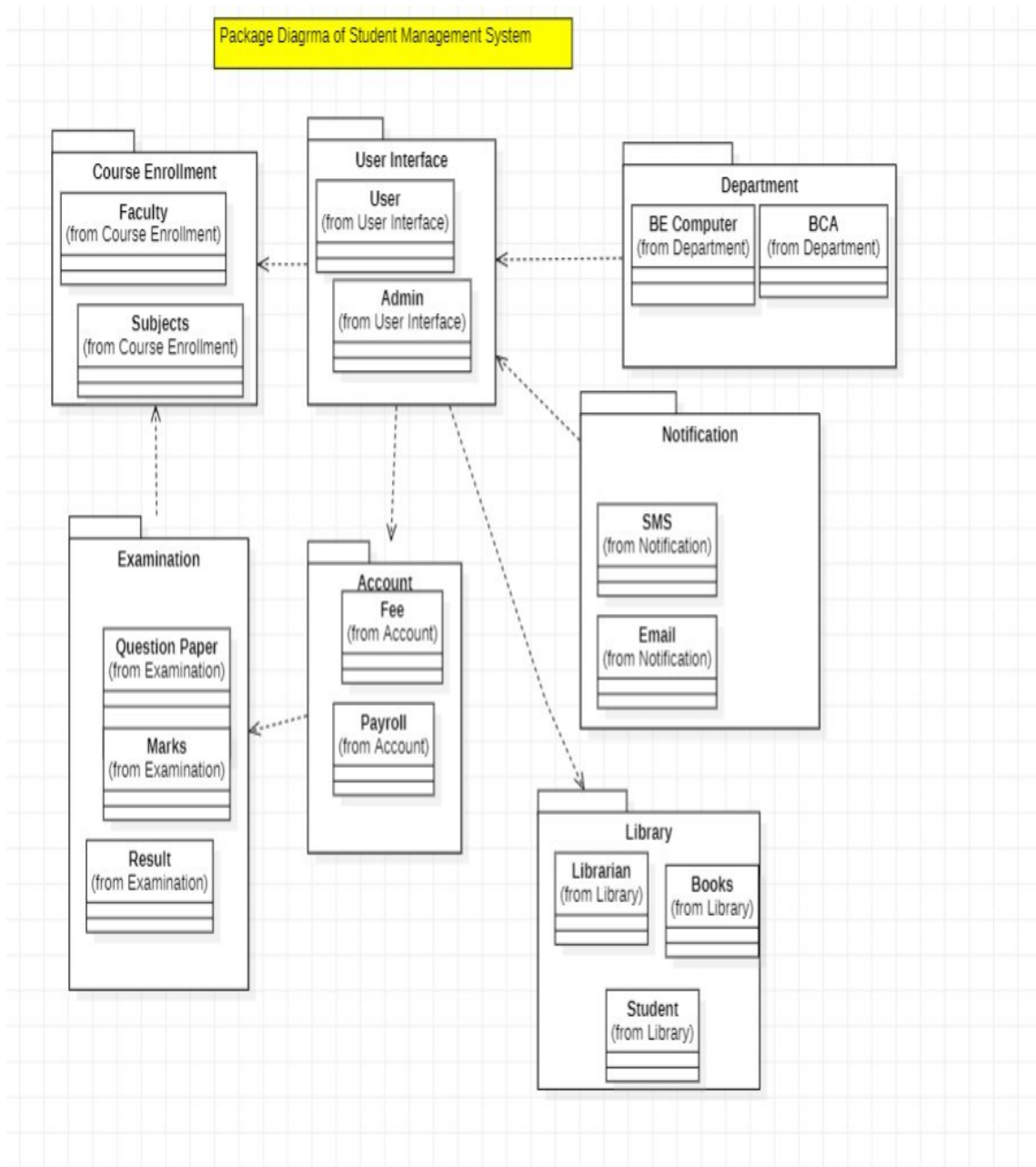
Conclusion

In this way we have learned to design Communication Diagram.

LAB 10: DESIGNING PACKAGE DIAGRAM USING CASE TOOLS

Introduction:

Package Diagram: A package diagram is a type of structural diagram in UML that organizes and groups related classes and components into packages.



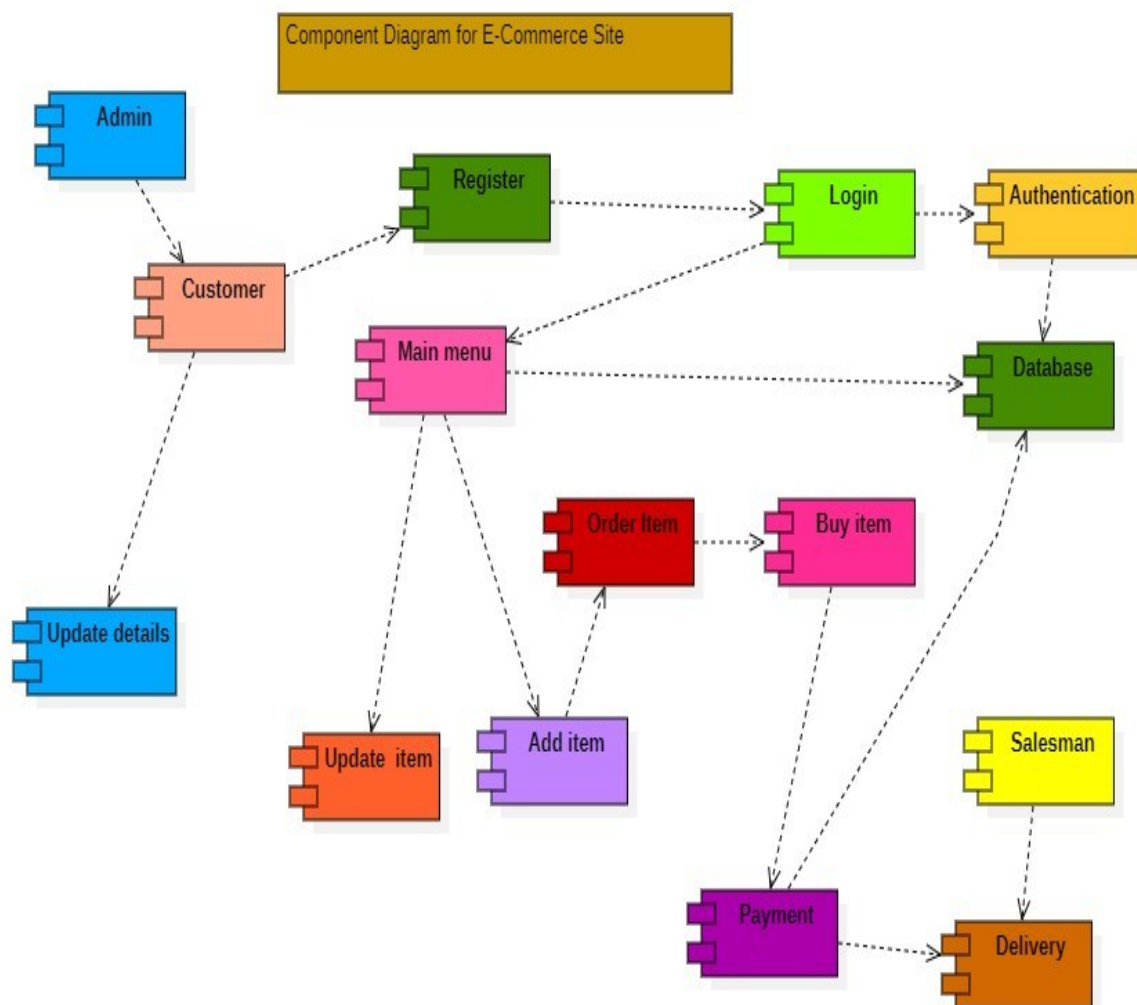
Conclusion

In this way we have learned to design Package Diagram.

LAB 11: DESIGNING COMPONENT DIAGRAM USING CASE TOOLS

Introduction

Component Diagram: It is a UML structural diagram that visualizes a system's physical architecture by showing its software components (like databases, libraries, or executables) and the relationships (dependences, interfaces) between them, illustrating how they connect and interact to form larger systems, often used for high-level design and understanding system structure.



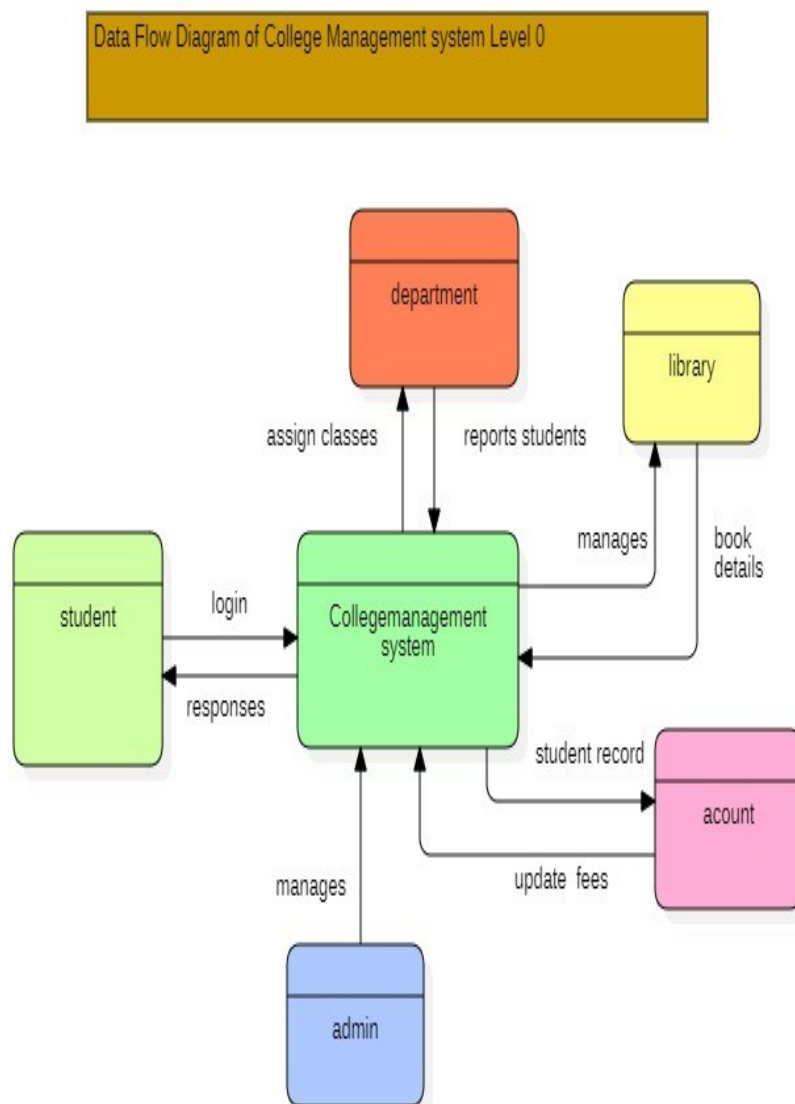
Conclusion

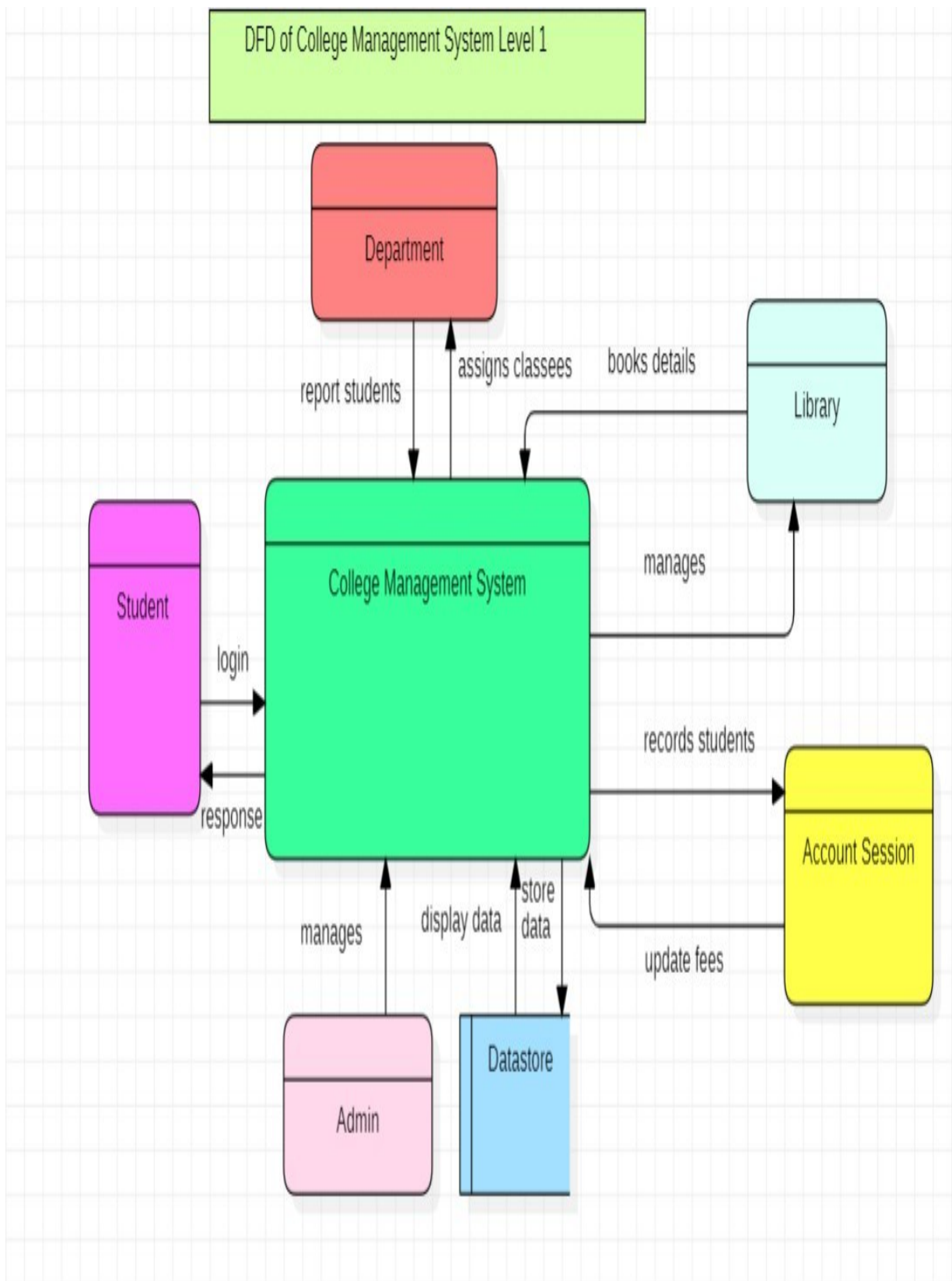
In this way we have to learn to design Component Diagram.

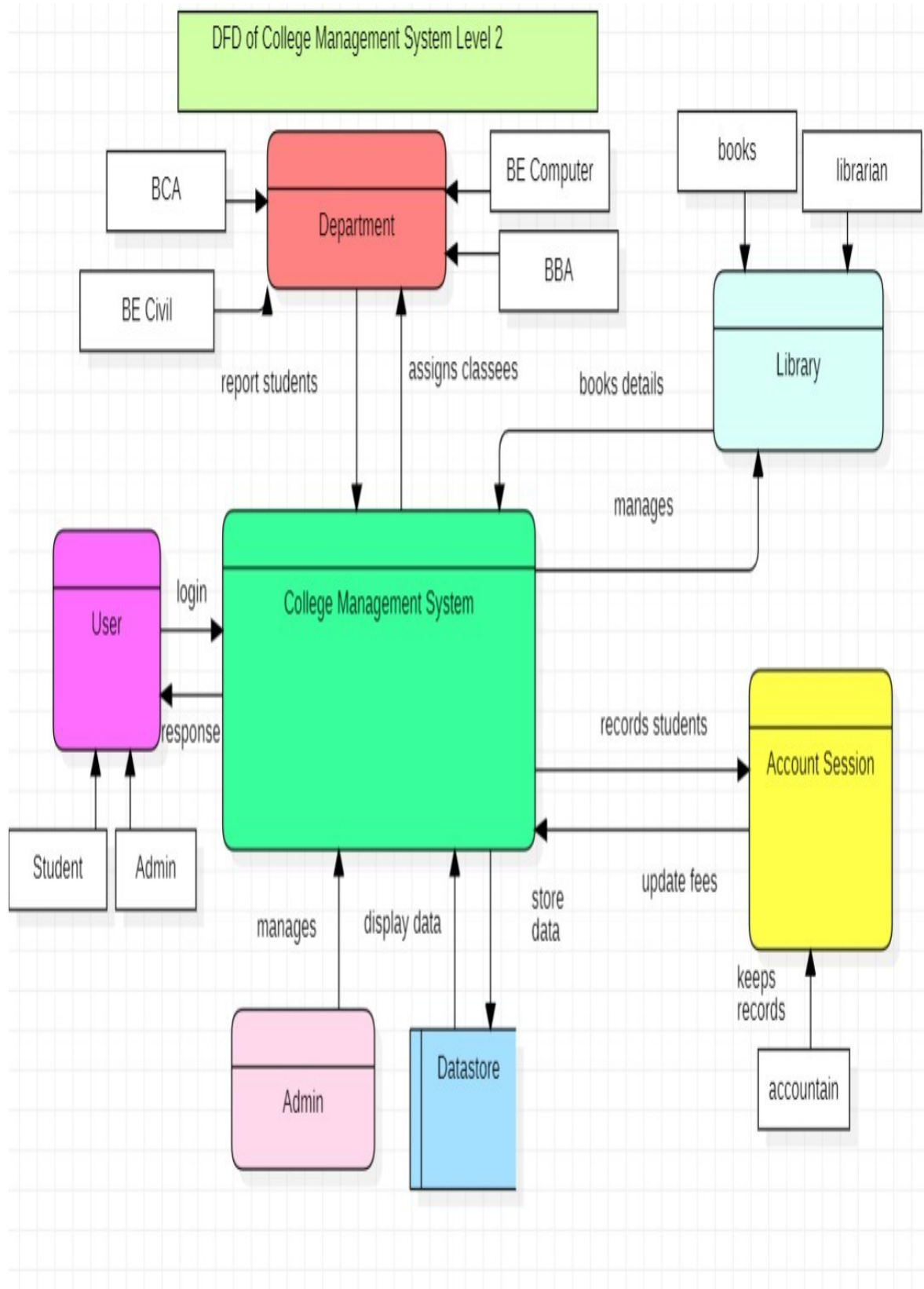
Lab 12: DESIGNING DATA FLOW DIAGRAM USING CASE TOOLS

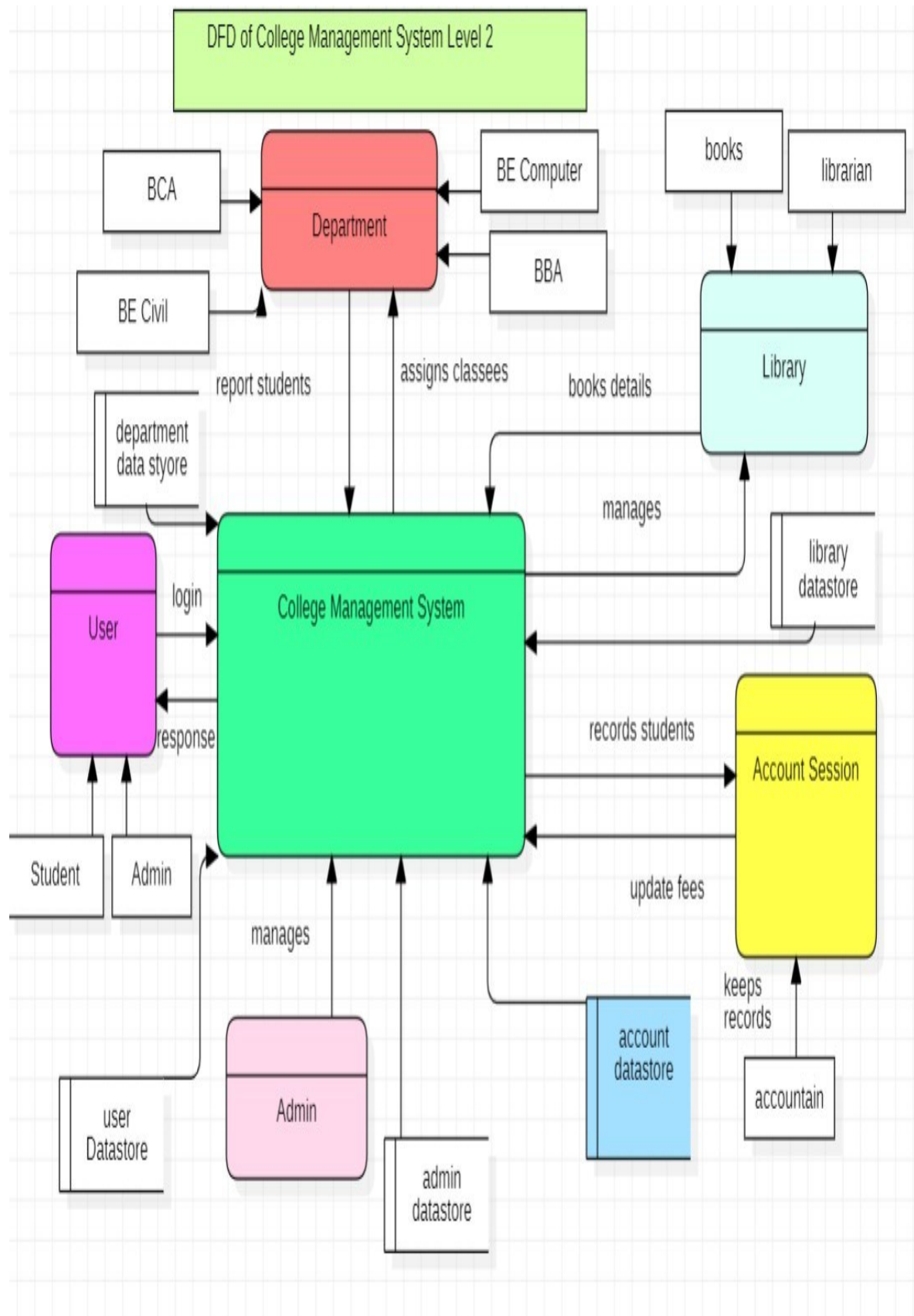
Introduction

Data Flow Diagram: It is a visual model that maps how data moves through a system, showing its sources, destinations, processes, and storage locations, using symbols for entities, processes, data flows, and data stores. It has different levels like Level 0, level 1, level 2, level 3 and so on.









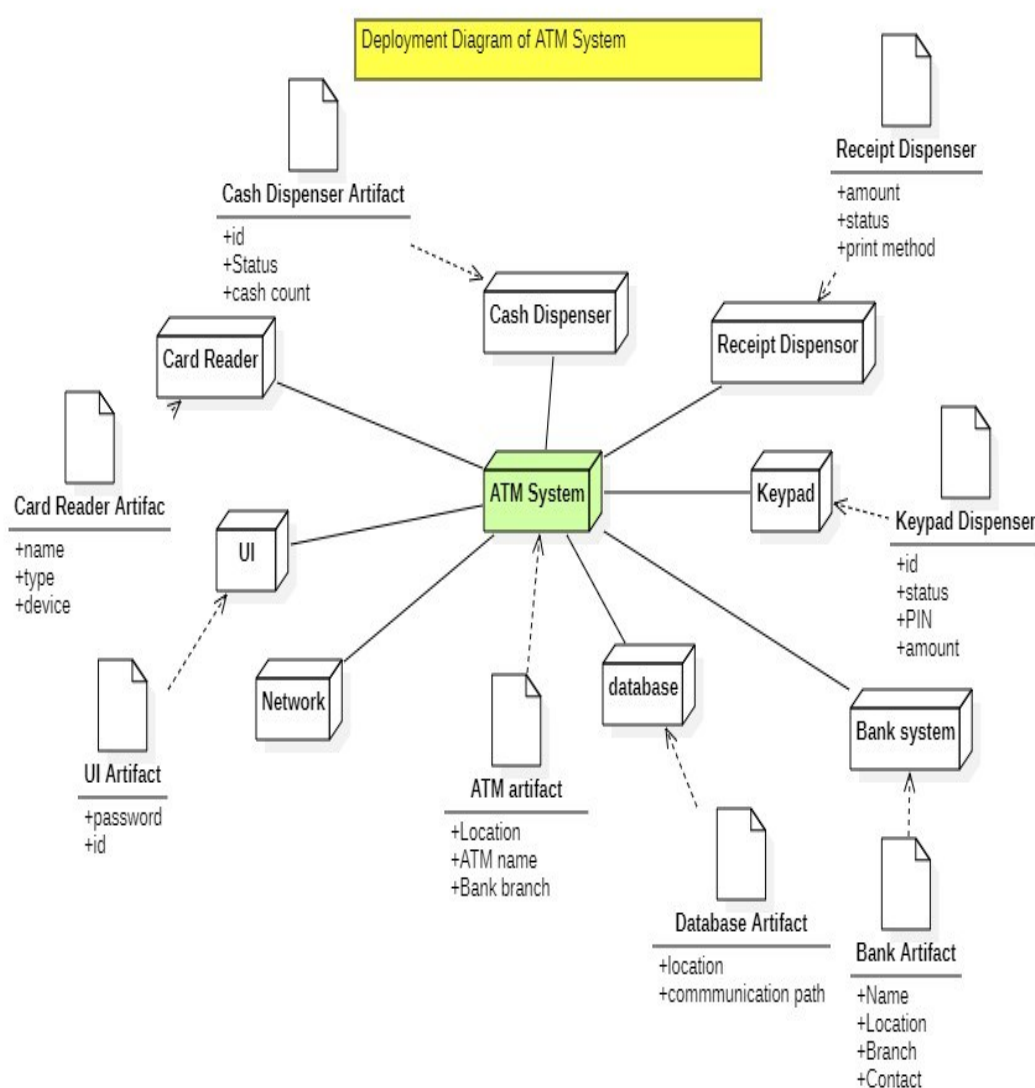
Conclusion

In this way we have learned to design Data Flow Diagrams.

LAB 13: DESIGNING DEPLOYMENT DIAGRAM USING CASE TOOLS

Introduction

Deployment Diagram: A deployment diagram is a part of UML that defines the physical architecture of a system, showing how software components (artifacts) are mapped onto hardware (nodes) like servers, devices, and execution environments, illustrating their distribution and communication paths in a real-world setting.



Conclusion

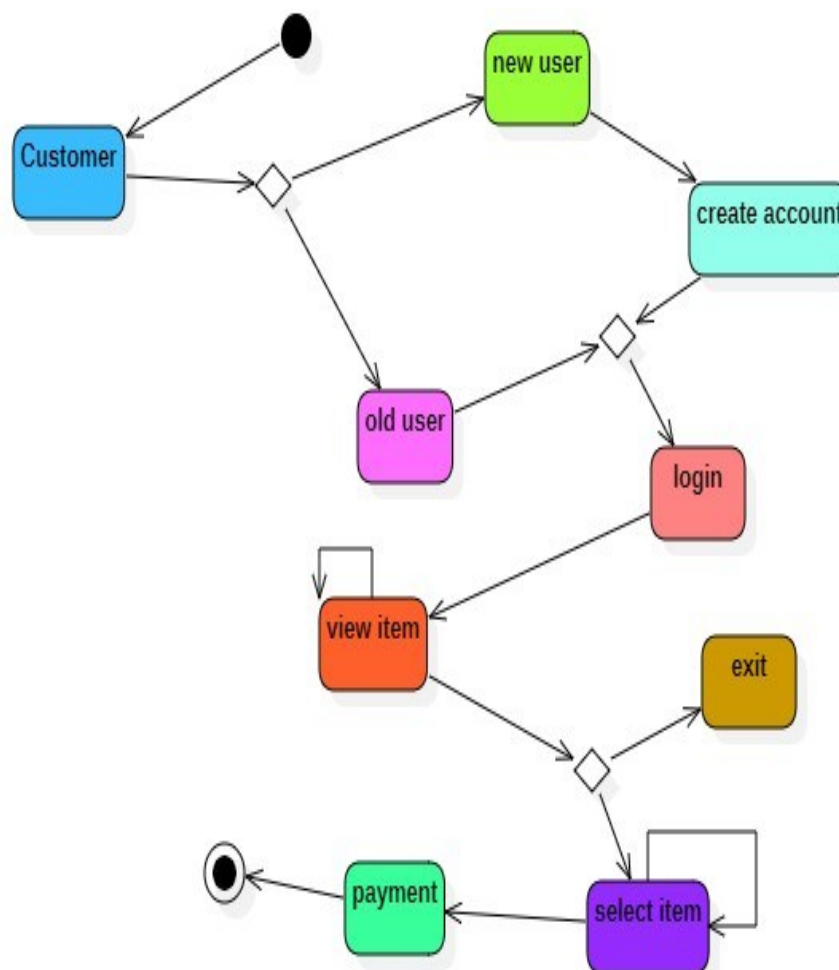
In this way we have learned to design deployment diagrams.

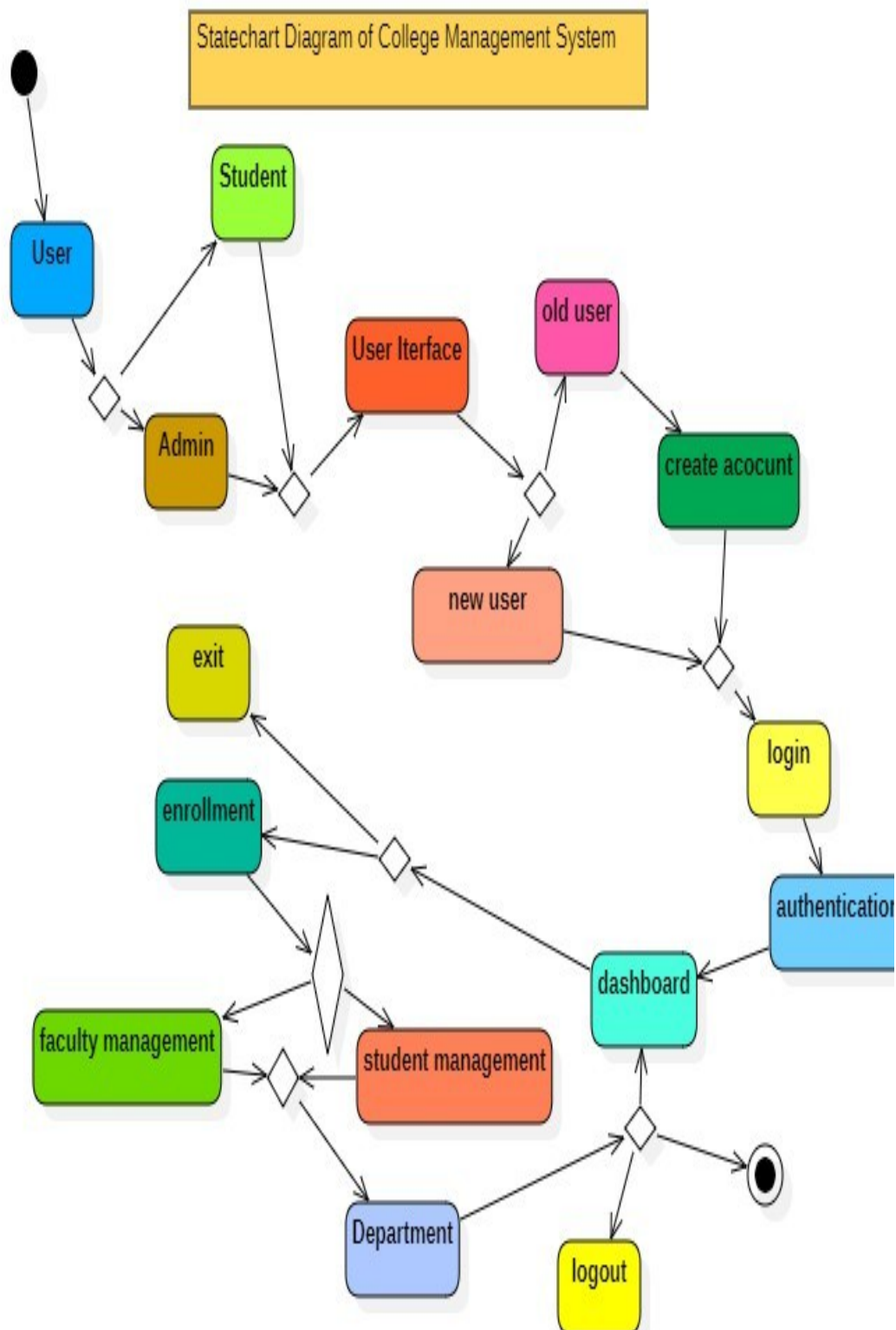
LAB 14 :DESIGNING STATECHART DIAGRAM USING CASE TOOLS

Introduction

Statechart Diagram: A state chart diagram is UML behavioral diagram that models an object's life cycle, showing its different states, events that trigger transitions between states, and the actions performed during those changes, focusing on an object's dynamic behavior from creation to termination.

StatechartDiagram of Online Shopping System





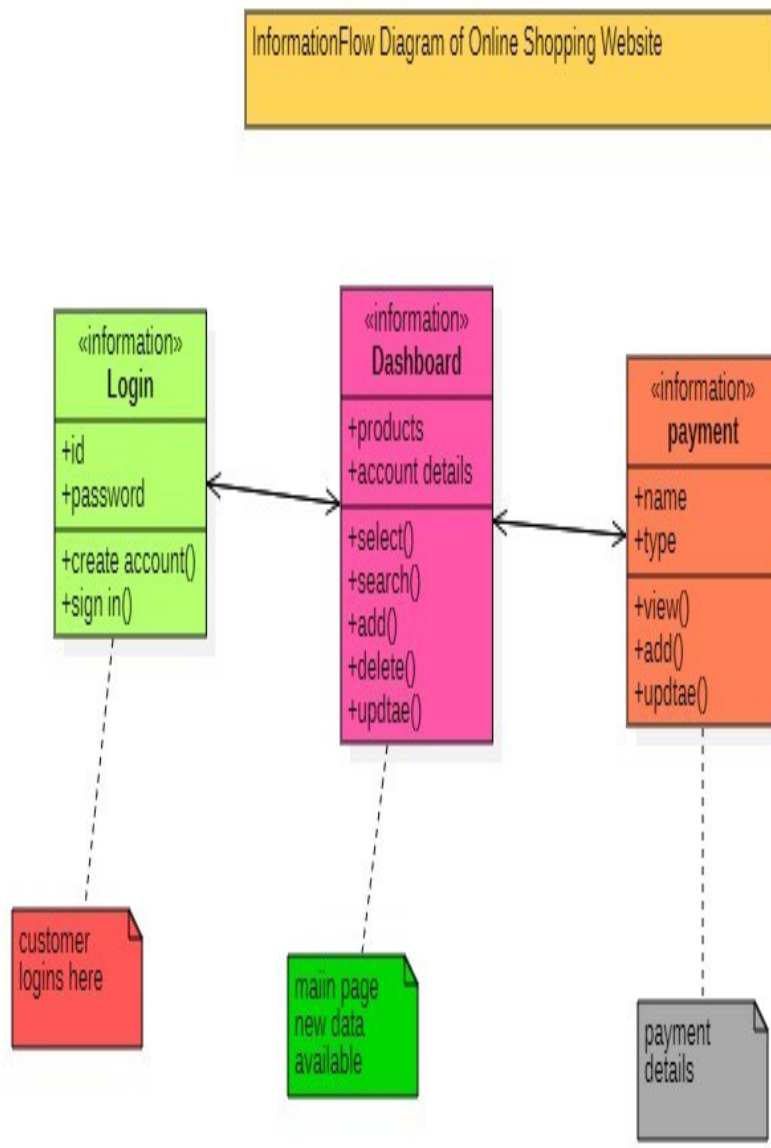
Conclusion

In this way we have learned to design Statechart Diagram.

Lab 15: DESIGNING INFORMATION FLOW DIAGRAM USING CASE TOOLS

Introduction

Information Flow Diagram: It visually maps how information flows between sources, systems, or people within an organization or process.



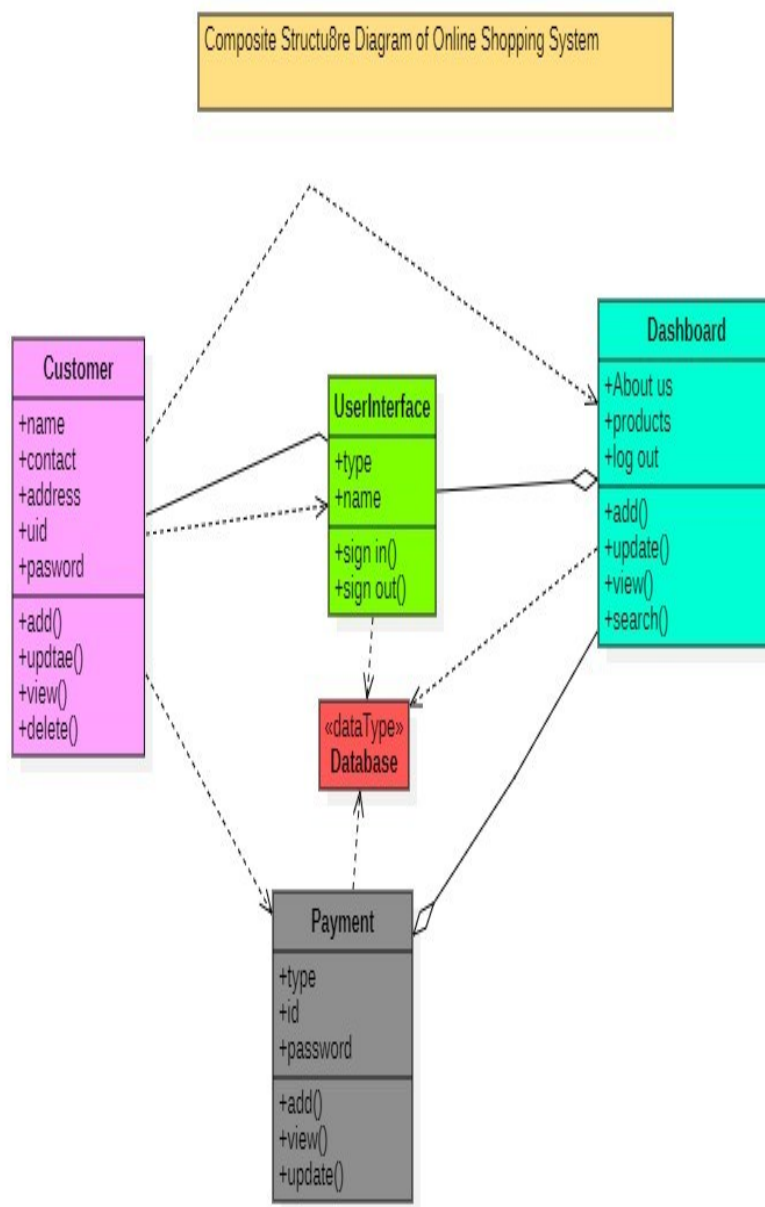
Conclusion

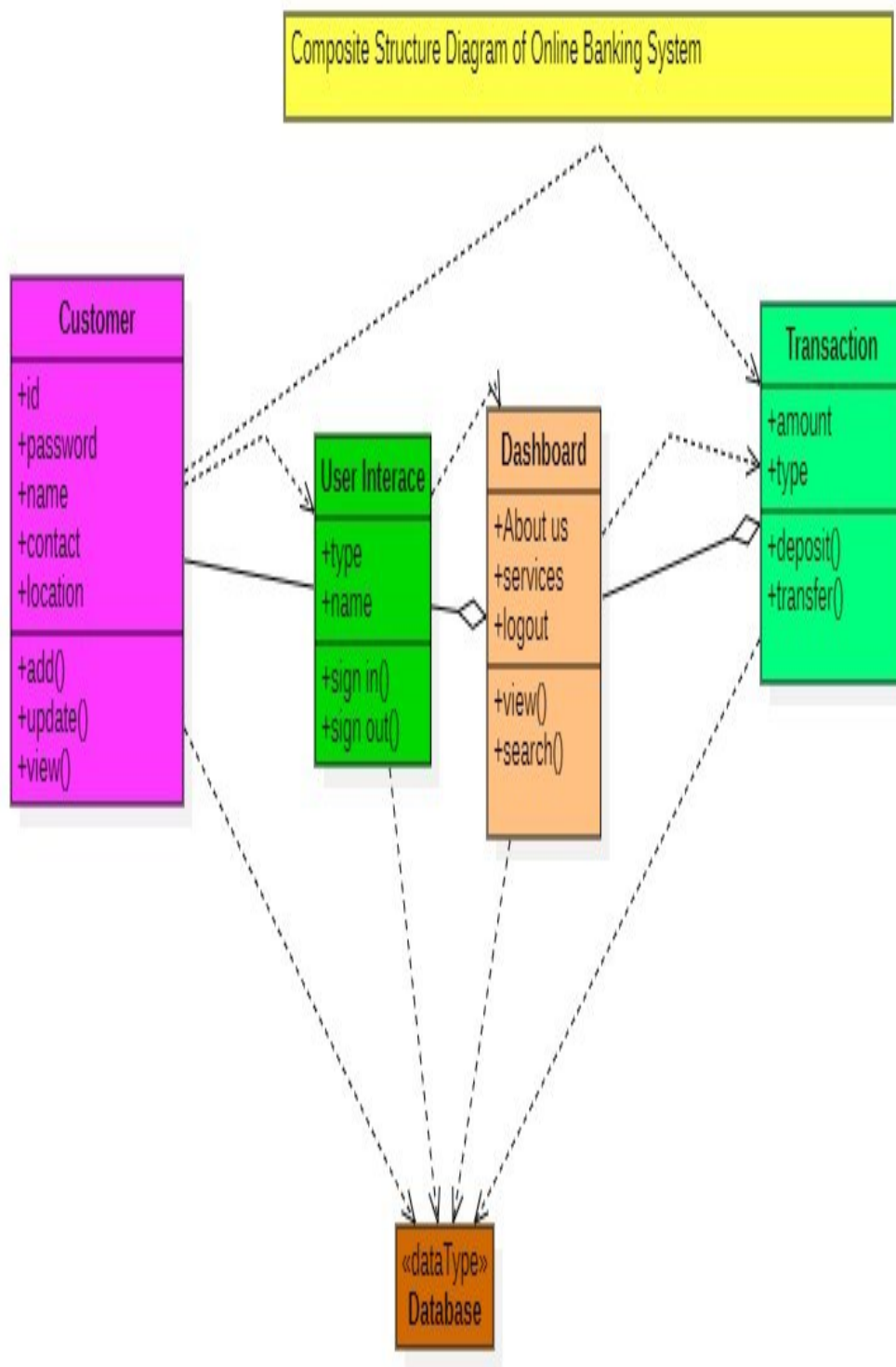
In this way we have learned to design Information Flow Diagram.

Lab 16: DESIGNING COMPOSITE STRUCTURE DIAGRAM USING CASE TOOLS

Introduction

Composite Structure Diagram: It is a UML that shows the internal structure of a classifier by modeling its constituent parts, their roles, interaction points and the connectors that link them, revealing how they collaborate to achieve the classifier's overall behavior.





Conclusion

In this way we have learned to design Composite Structure Diagram.

LAB 17: TEST PHASE DEVELOPMENT

Introduction

Test Phase Development: The testing phase development is a critical, multi-stage process designed to verify software functionality, performance, and quality, ensuring it meets user requirements before launch.

Test Phase Development for Mobile Application Integration					
S.N	Test Phase	Objective	Entry Criteria	Exit Criteria	Deliverables
1	Assembly Integration	To validate the integration between application and web	Mobile application should integrate with web	Errors and logged, mobile component are checked	test script automated documentation
2	Application validation	To verify the Validation of Mobile Application	Mobile application should be validated	Errors and logged, mobile validation are done	test script automated documentation
3	Component Testing	To check that all the components are working efficiently	Components of Mobile application should be well functioning	Errors and logged, mobile component are checked	test script automated documentation
4	Regression	To perform Mobile Application verification and validation regeressly	Mobile application should be Regeressly Tested	Errors and logged, mobile component are checked	test script automated documentation

Conclusion

In this way we have learned to develop test phase development.

LAB 18:TEST CASE DEVELOPMENT

Introduction

Test Case Development: It is the phase in the Software Testing Life Cycle (STLC) where QA teams create detailed, documented sets of conditions, inputs, steps, and expected results to verify software functionality against requirements.

Project:Student Management System							
Module:Login							
SN	Test Case Id	Test Scenarios	Test Steps	Test Data	Expected Outcome	Actual Outcome	Status
1	TC-01	Login Validation	Login the system using id and password	Userid:test@gmail.com Password:tset@123	Validation successful	Login successfully	pass
2	TC-02	User Validation	Login the system using id and password	Userid:test@gmail.com Password:tset@123	User id should be validated	Userid is validated	pass
3	TC-03	Password Validation	Login the system using id and password	Userid:test@gmail.com Password:tset@123	Password should be valid	Invalid password	fail
4	TC-04	Textbox Validation	Login the system using id and password	Do not entry any data	Test box should show as required field	Textbox is showing required messages	pass
5	TC-05	Button Validation	Login the system using id and password	Userid:test@gmail.com Password:tset@123	Button should direct to main page	Login button is working smoothly	pass
6	TC-06	Forget Password Validation	Create new password and confirm password	New Password:se@12345 Confirm password:se@12345	new password should be created	password crettaed	pass
7	TC-07	Show Password Validation	Click on show password button	visibility of eye icon	Password should be show	password is seen	pass

Conclusion

In this way we have learned to design Test Case Development.

LAB 19:TEST REPORT

Introduction

Test Report: It is the final state of test where all details about test is documented.

SN	Test Case id	Test Scenario	Test Step	Test Data	Expected Output	Actual Output	Status	Tester	Remarks
1	TC-01	Email Validation	enter valid email and verify with OTP	email:test@gmail.com and OTP	email verified	Email verified successfully	pass	Er.Deepa Paneru	
2	TC-02	User Validation	enter username,date of birth,gender and all details	Username :Paban@123	User is valid	user is created	pass	Er.Sandarv Lamichhane	
3	TC-03	Password Validation	Enter a strong password	Pssword:Eps@16%	invalid password	create a strong password	pass	Mr.Dipak Joshi	
4	TC-04	Login valiation	Enter username ,password	Username :Paban@123 Password:Eps@136%	Login successfully	Loged successful	pass	Ms.;Hemani Pant	
5	TC-05	Button Validation	Click te signup button	Username :Paban@123 Password:Eps@136%	Signup Button wortking smoothly	Button working correctly	pass	Mrs.Soniya Khadka	

fig: Test Case Execution Report

Project :Student Manageent System						
Module:Sign Up						
SN	Test Case Mdule	Bug id	Bug Types	Tester	Remarks	
1	Login Module	BG -04	Password is incorrect Message wasnot showing	Ms.Hemani Pant		
2	Signup Module	BG-05	signup is not successful alhtough correct password	Mr.Rakesh Sharma		
3	Email Module	BG-04	OTP I snot come while email is valid	Mrs.Santosh Jaishi		

Fig: Bug Report

Project :Student Manageent System								
Module:Sign Up								
SN	Testcase Module	No of Test Cases	Pass	Fail	Block	Pass Percentage	Fail Percentage	Status
1	Email Module	2	2	0		100%	0%	pass
2	Password Module	1	1	1		0%	100%	fail
3	Signup Module	1	1	0		100%	0%	pass
4	Login Module	2	2	0		100%	0%	fail

Fig: Test Report

Project :Student Management System								
Module:login								
S.N	Test Case ID	Test Scenario	Test Steps	Test Data	Expected Outcome	Actual Outcome	Status	Tester
1	TC-101	Login Validation	Enter valid username & password	username:user@test.com / password: pass123	Login successful	Login successful	pass	Bhuwan
2	TC-102	Invalid username	Enter invalid username	username:wrong@test.com / password: pass123	Error message shown	Error message shown	pass	Kaushal
3	TC-103	Invalid password	Enter wrong password	username:user@test.com / password: wrong	Error message shown	Error message not shown	fail	Urmila
4	TC-104	Empty fields	Click login without data	_____	Validation message shown	Validation message shown	pass	Prakash
5	TC-105	Login button	Click login button	valid credentials username:user@test.com / password: pass123	Redirect to dashboard	Page not redirected	fail	Nitu

Project :Student Management System								
Bug Report								
S.N	Bug ID	Module	Test Case ID	Bug Description	Severity	Status	Remarks	Tester
1	BG-01	Login	TC-103	Error message not shown for invalid password	Medium	Open	Validation message needs to be implemented	Dinesh
2	BG-02	Login	TC-105	Login button does not redirect user to dashboard	High	Open	Navigation issue found after login	Pradeep
3	BG-03	Signup	TC-S-03	Password mismatch warning not displayed	Medium	Open	Password confirmation logic missing	Rachana
4	BG-04	Signup	TC-S-05	Signup button not redirecting to login page	High	Open	Page redirection needs fixing	Sandarv

Project :Student Management System									
Test Report									
S.N	Module	Total Test Cases	Passed	Failed	Blocked	Pass %	Fail %	Status	Tester
1	Login Module	5	3	2	0	60%	40%	Needs Fix	Deepa
2	Signup Module	5	3	2	0	60%	40%	Needs Fix	Paban
	Overall	10	6	4	0	60%	40%	Improvement Required	

Conclusion

In this way we have learned to prepare test reports.

LAB 19: INTRODUCTION TO SOFTWARE MANAGEMENT TOOL JIRA

Introduction

Jira is a project management and workflow automation tool designed to streamline collaboration and productivity within teams. It is widely used in software engineering for task tracking, team coordination, and automating repetitive workflows. Jira integrates with development platforms and tools such as GitHub, allowing developers to manage issues, pull requests, and project milestones effectively.

Key features of Jira include:

- Workflow automation using triggers and actions.
- Customizable project boards.
- Integration with version control systems.
- Advanced reporting and analytics.

Materials and Tools Required

A computer with an active internet connection.

- Access to the Jira tool (free or licensed version).
- A GitHub account (optional, for integration purposes).
- A browser or Jira desktop application.

When to Use Jira

Jira should be used when:

- Managing complex projects with multiple team members.
- Automating repetitive tasks in workflows to save time.

- Collaborating across departments or remote teams.
- Monitoring project progress and generating detailed reports.
- Tracking tasks, issues, and pull requests in software development.

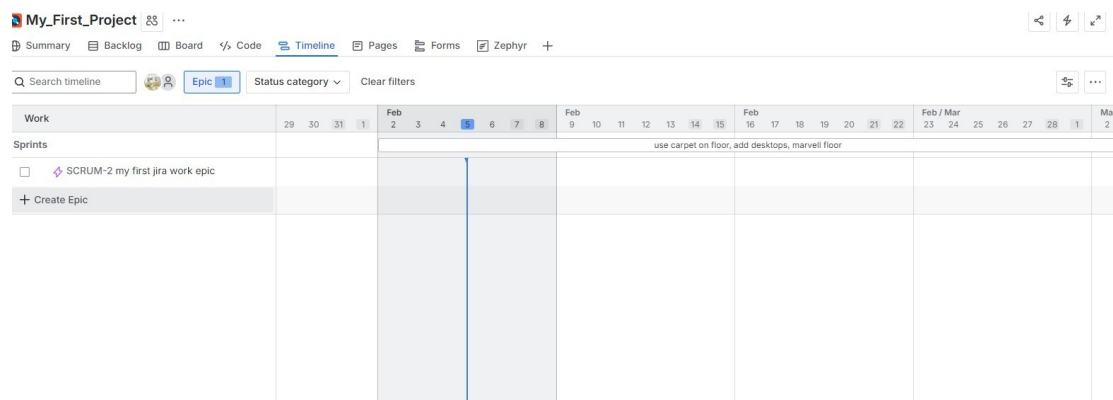
Advantages of Zira

- User-Friendly Interface: Easy to navigate and use.
- Automation: Saves time by automating repetitive processes.
- Integration: Works with tools like GitHub and Slack for a unified experience .
- Customization: Highly customizable boards and workflows to fit team needs.
- Improved Collaboration: Facilitates better communication and task delegation.
- Scalability: Suitable for teams of all sizes.

Applications of Zira

- Software Development: Managing issues, pull requests, and project milestones.
- Marketing Campaigns: Tracking tasks and deadlines.
- Agile Project Management: Using Kanban and Scrum methodologies.
- Human Resources: Onboarding workflows and employee task management.
- Education: Managing projects and assignments for academic purposes.

We just learn to create a project and epic on the first day of Jira as shown in figure.



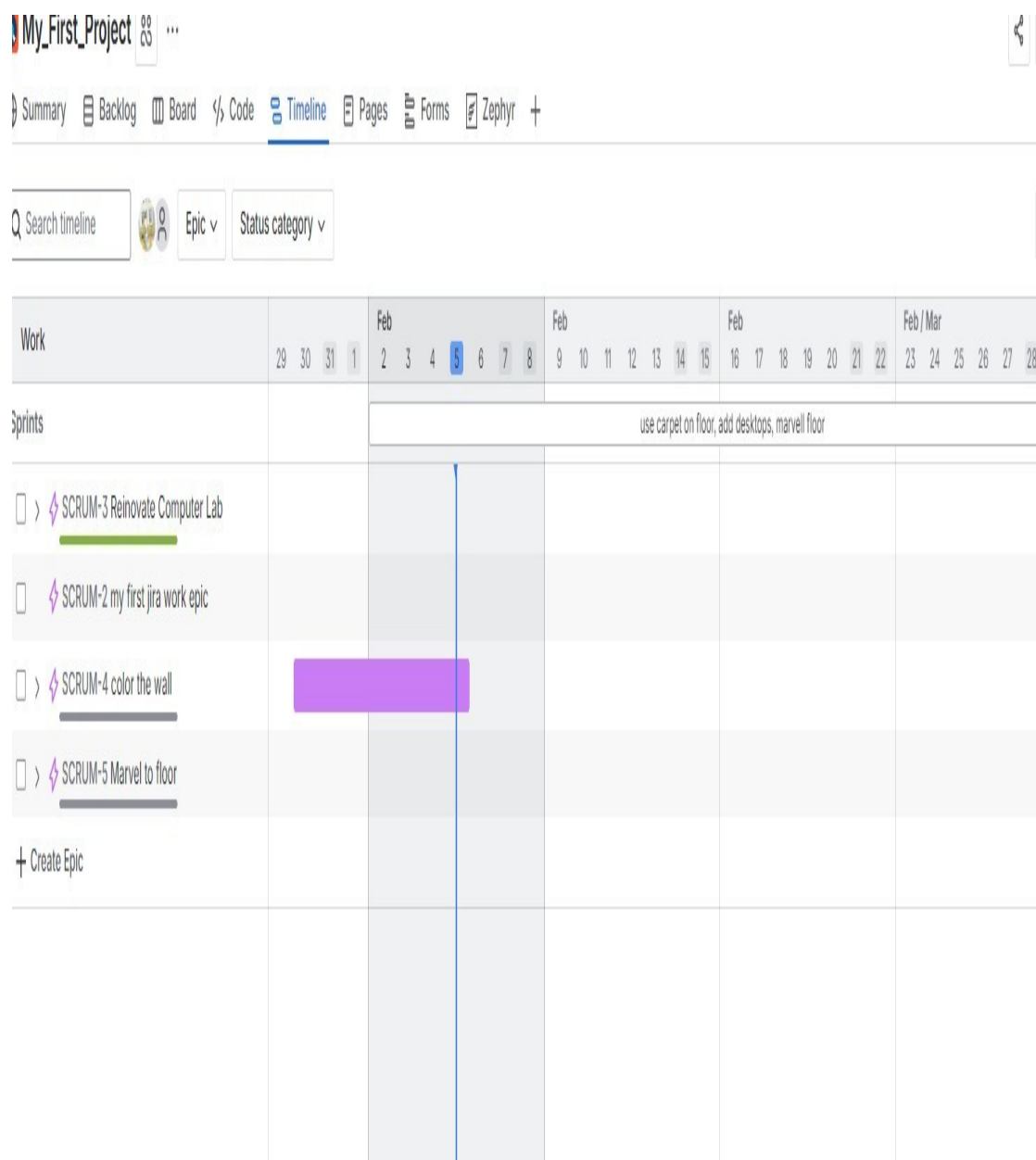


Fig: Creating multiple epics in first project.

Conclusion

In this way we know the interface of Jira.

LAB 20: CREATING SPRINT AND PREPARE REPORT S OF

TASKS USING JIRA

Introduction

Sprint: A **sprint** is a fixed, time-boxed period (typically 1-4 weeks) where an agile team commits to completing a specific set of tasks or user stories from the product backlog.

Steps to create a sprint:

1. Create a new project if you have not a Project or choose a project if you have
2. Create epics on this project by going timeline and add task or bugs to them
3. Create sprint using epics.
4. Activate Report options going to space settings 5. Choose a type of report and see the result.

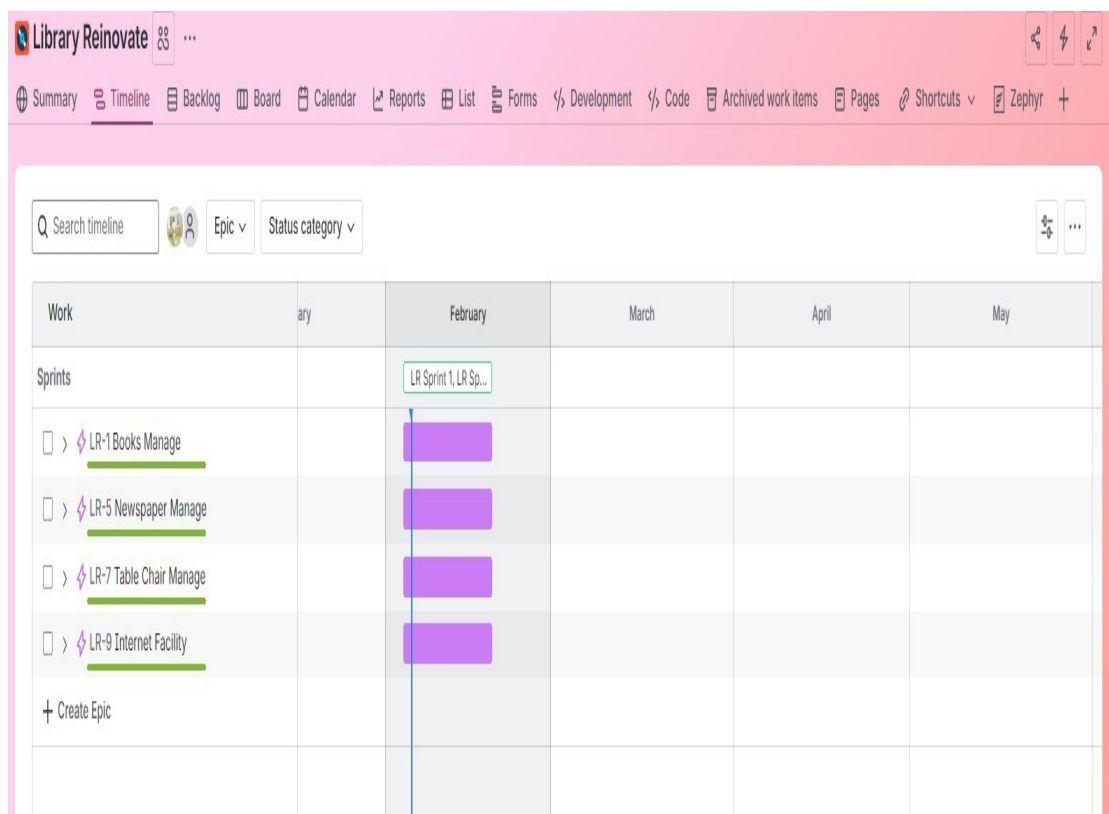


Fig: Creating different Sprints

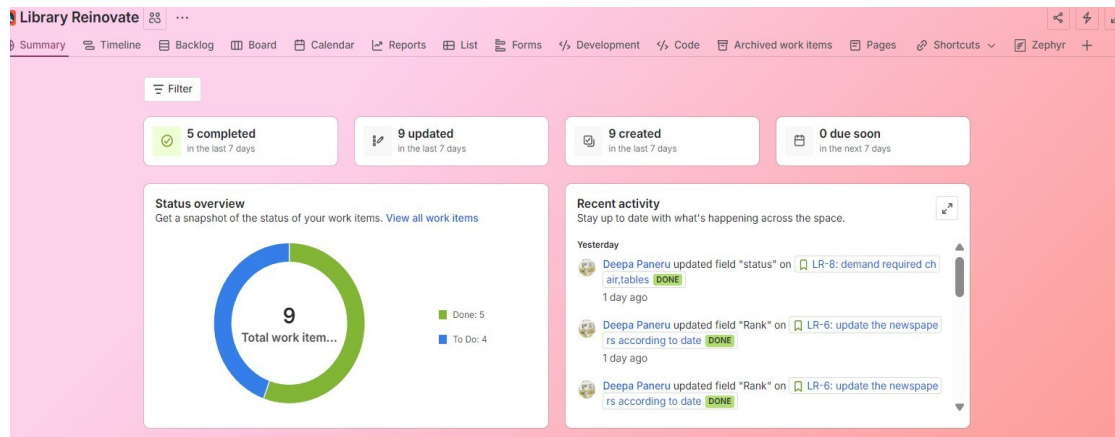


Fig: Sprint successful

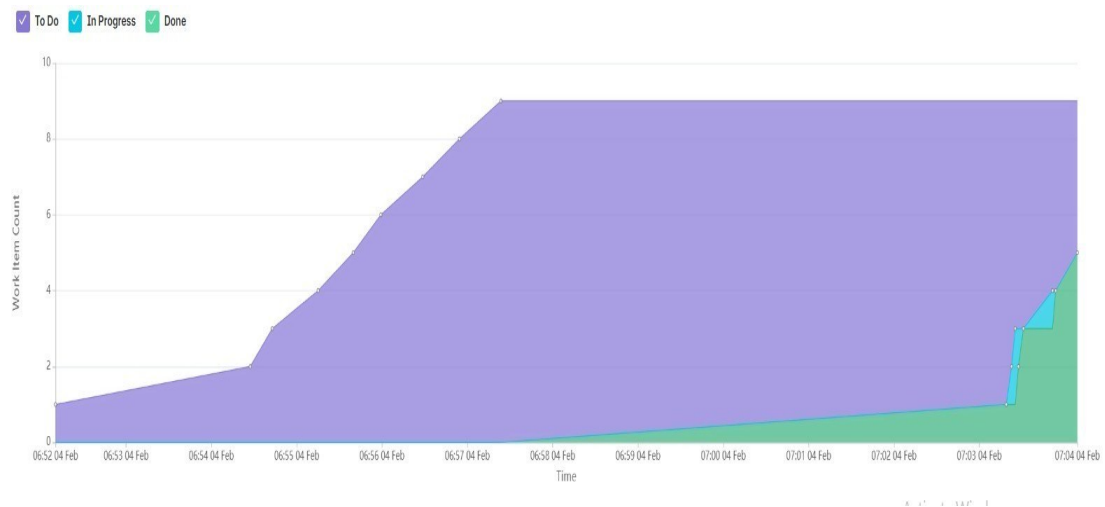


Fig: Tasks Report of all tasks



Fig: Task Report on progress tasks

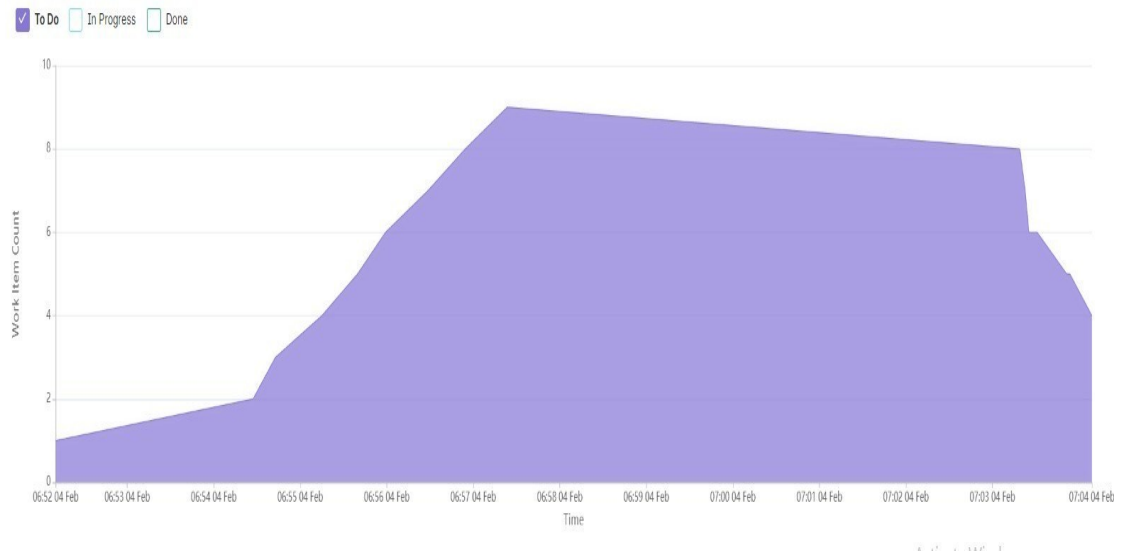


Fig: Task Report of to do tasks

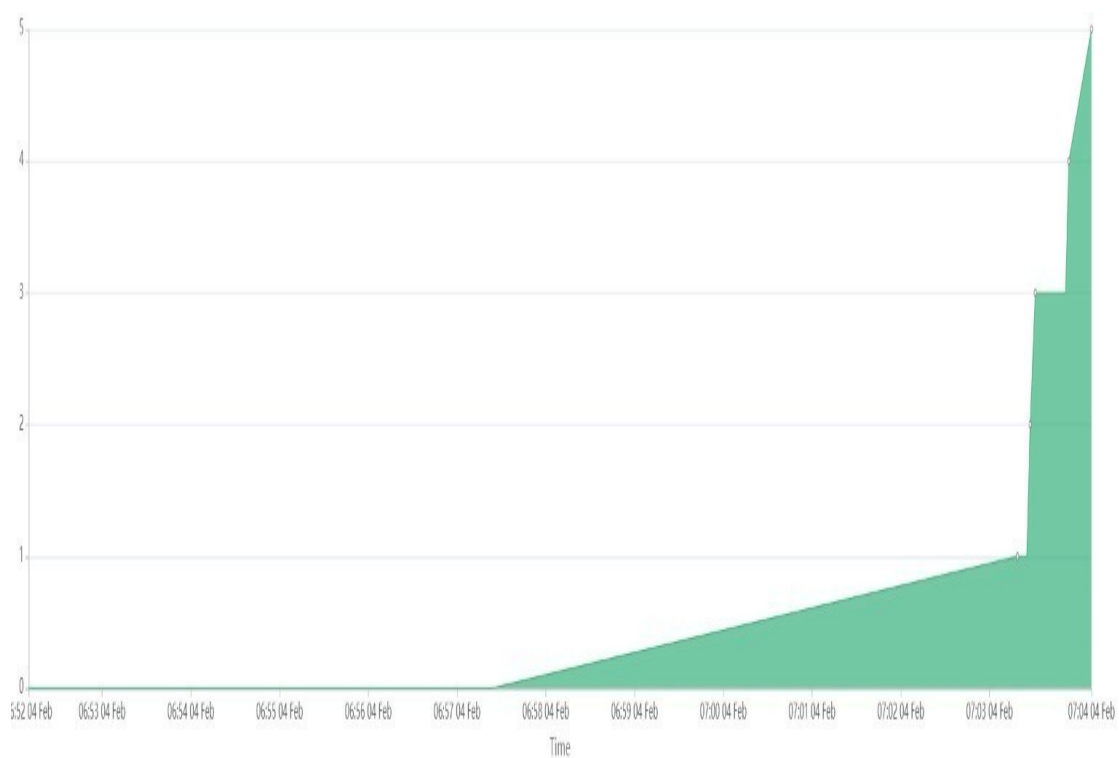


Fig: Task Report on tasks done

Conclusion

In this way we have learned to create sprint and prepare tasks report.

LAB 21: INTEGRATING ZEPHYR WITH JIRA AND TEST CASE DEVELOPMENT

Introduction

Zephyr: Zephyr is a popular, comprehensive test management designed to manage the entire testing lifecycle like planning, authoring, executing, and reporting directly within *the Jira*. It enables teams to create, organize, and link test cases to requirements and user stories, providing real-time quality metrics and traceability within Jira Interface.

Steps to integrate Zephyr with Jira

1. Go to menu bar and click on Apps.
2. Then search Zephyr and click on it.
3. Choose to try to free and choose standard version
4. Then install it and it comes in the top navigation.

Steps to create Testcase Development with Zephyr:

1. Go to Test Cases and click on new Test Case.
2. Add test cases and fill the category like Test steps, Test data and expected Output.
3. Add the test cases how many you want to.
4. Then go to back and click on a test case and then you see details about that
5. Then click on Test Script and click to automate test .
6. Then fill the website of Orangehrm and run the test .

Student Management System / Test Cases / Forget_Password (LR-T3)

Go back

Version 1.0

Details Test Script Traceability Execution History

Description

Name*
Forget_Password

Objective
regenerate password if user forget the previous password

Precondition
email linked to account is necessary

Details

Status Approved	Priority High	Component None
Owner Deepa Paneru	Estimated run time 1hmm	Folder None

Activate Windows
Go to Settings to activate Windows.

Fig: Test Case Description

Spaces

Student Management System

Summary Timeline Backlog Board Calendar Reports List Forms % Development % Code Zephyr More 3 +

SMARTBEAR Zephyr Configuration

Test Cases Test Cycles Test Plans Reports

+ New Folder Q ... + New Test Case Archive Clone More

Search... Q Filters

All test cases (1)

P	Key	V	Name	Status	R
LR-T3	1.0		Forget_Password	APPROVED	
LR-T1	1.0		Login validation	APPROVED	
LR-T4	1.0		Login_Module	APPROVED	
LR-T2	1.0		Signup_Validation	APPROVED	

Fig: Test Cases List

Student Management System / Test Cases / Forget_Password (LR-T3)

Go back

Version 1.0

Details Test Script Traceability Execution History

Type: Step by Step Automate Test

Data type: None

Steps (8)

STEP	TEST DATA	EXPECTED RESULT
1 Open the interface	Interface is suitable or not	Verify that the interface is working efficiently
2 Click on the Forget Password button	None	Verify that the email receives an OTP
3 Input the OTP into the input field	456789	None
4 Input the new password into the password field	password@123	Validate that the password is accepted

Activate Windows
Go to Settings to activate Windows.

Fig: TestScript of Forgrt Password

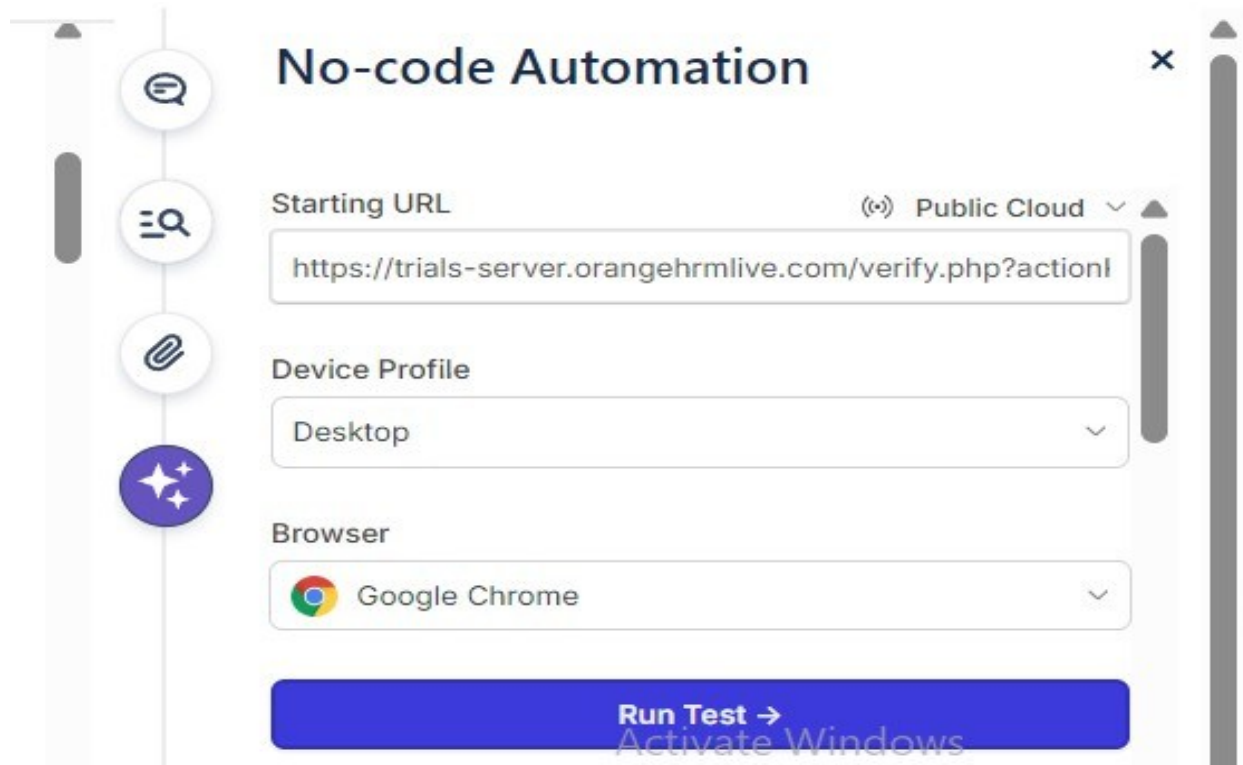


Fig:Automation Test Platform

Conclusion

In this way we have learned about Zephyr and Test Case Development.